

RESEARCH PAPER

An automated functional testing approach for virtual reality applications

Alinne C. Corrêa Souza¹  | Fátima L. S. Nunes² | Márcio E. Delamaro³

¹Technological Federal University of Paraná,
Dois Vizinhos, Brazil

²School of Arts, Sciences and Humanities,
University of São Paulo, São Paulo, Brazil

³Institute of Mathematical and Computer
Sciences, University of São Paulo, São Carlos,
Brazil

Correspondence

Alinne Souza, Universidade Tecnológica
Federal do Paraná, Dois Vizinhos, Paraná,
Brazil.

Email: alinne.santos7@gmail.com

Summary

Software testing is regarded as an important method for fault revealing. Despite this advantage, it has been poorly used within the scope of virtual reality (VR) applications because they are highly complex and have peculiar features. Most testing performed of this VR applications are usability, which is conducted manually and only at final of the development process. Although some works try to propose criteria for this domain, there are no approaches that automatize the generation of test data from requirements specification in the VR domain. This paper proposes an approach called virtual reality—requirements specification and testing (VR-ReST) to assist the requirements specification through a semiformal language and uses structural test criteria to generate test requirements and test data automatically for VR applications using scene graph concepts. The paper also examines the empirical results concerning the cost-effectiveness of the approach for three different VR applications through two experiments. Mutation testing was used to evaluate effectiveness. We found that the approach achieved a high mutation score outperforming random testing, by 20%, on average. Our results also demonstrate that the approach is promising since it assists in writing and validating the requirements, as well as in reducing the risks of requirement specification by adopting a semiformal language.

KEYWORDS

requirements engineering, software testing, virtual reality

1 | INTRODUCTION

For several years, effort has been put into this research area in tasks such as development of techniques, testing criteria [1, 2], and automation of the testing activity [3, 4, 5]. Some studies are dedicated to changing, adapting, and improving traditional techniques to accommodate peculiarities of specific software domains, such as WEB applications [6, 7, 8], database applications [9, 10], and GUI testing [11, 12]. Our interest here lies in the domain of virtual reality (VR) applications, with their specific characteristics and high complexity.

VR systems are three-dimensional (3D) virtual environments (VEs), as opposed to traditional two-dimensional environments considered over the years in computational applications. Features such as interaction, immersion, and the importance of using conventional and unconventional input/output devices make VR programs highly complex [13]. Such features also add difficulty to the process of software development, in particular, requirements specification and software testing activities.

VR applications have a powerful interface that allows advanced human-computer interaction by using human senses. Such interfaces enable the user to immerse, navigate, and interact in a three-dimensional environment using multi-sensory devices [14]. VR is being increasingly used, and it is regarded as a technology that considerably contributed to several application areas such as medicine [15, 16], archeology [17], entertainment [18, 19], and education [20].

VR applications, like other complex systems, may involve features such as parallelism and synchronization of actions to respond to several devices that can be coupled to the application. Additionally, to be characterized as a VR application, interaction in real time in 3D environment is mandatory. Although most of real systems are interactive, the third dimension requires more complex techniques and specific algorithms during the interaction. For example, an important and specific computation that must be executed due to an interaction is defining which part of the 3D object must be represented in the scene. This occurs because usually, exhibition displays are two-dimensional and, thus, the portion that

will be visualized must be computed considering parameters such as camera position, object's features and behaviors, and user's actions. These parameters and other specific features make it harder to develop VR applications and require specific approaches during the testing.

In addition, specifying requirements of VR applications is also a complex process, because they can comprise the human senses (touch, sight, hearing, and perception) and the virtual object behaviors. These behaviors can involve adapting complex algorithms for simulating physical characteristics of objects, such as adequate responses for collision detection and deformation. Such specifications depend on a combination of different requirements arising from various scenarios with heterogeneous features and a wide range of interactions [13]. Considering objects, behaviors, and users, it must be highlighted that (1) responses of the system are highly users-dependent and objects-dependent and (2) it is too difficult to reproduce an exact interaction previously executed by a user. Together to the features cited before, this compose the motivation to develop new approaches for testing VR applications [21].

Some studies presented approaches [22, 23, 24], models [14, 25], methodology [26], and multicriteria assessment of usability [27] to improve VR application development processes. However, most of these approaches are related to just part of the requirement engineering (RE) process and are usually decoupled from the software testing activity. To promote such integration of requirements specification and testing of VR systems, the current paper proposes a semiformalized approach to specify requirements and, then, use structural criteria to exercise those requirements.

In general, the tests on VR applications are more specific and performed manually only after the development, making the process unreliable and increasing the cost of fault fixing. In many cases, VR applications involve human senses, so the tests to be performed can be considered subjective when compared with traditional applications. On the other hand, testing is essential due to the technological issues, the quality of the application, and the psychological and social impact. In this context, tests in VR applications should consider: (1) the performance of the devices and general ergonomic features; (2) the interface aspects and their influence on interaction; (3) the discrimination of the visual aspects; (4) auditory issues; (5) the tact and strength issues; and (6) behavior, performance, and consequences of the simulation.

A way to achieve a level of trust and quality is to employ techniques and test criteria to assist the generation of test requirements that must be covered during the test activity. The practical use of a criterion was only addressed in the work by Bezerra et al [28], which proposed four testing criteria (All-Nodes-Leaves, All-Nodes-Intermediary All-Paths-Ascendants, and All-Paths-Descendants) for VR applications based on structural testing criteria. Some studies explore different aspects of software testing related to VR applications such as supporting testing of VR interfaces [29] and usability testing of VR-based learning environments [30]. To our knowledge, there is still no approach involving functional testing activity used before and during the development process of VR applications.

Formal methods allow system analysis and test generation from specifications. This provides an early feedback on systems behavior. In contrast, methods and tools have been developed to analyze systems using their specification (to prevent unexpected behaviors) and to generate test data (to guarantee the fitness of the implementation to the model). Despite benefits of test data generation, we did not identify studies that applied formal methods to automatically generate test cases for VR domain.

From this perspective, the approach presented here, called virtual reality—requirements specification and testing (VR-ReST), aims at generating test data for VR applications, based on their rigorous requirement specifications. Our approach is based on two elements: (a) a semiformal requirements specification that aims to determine the functionalities to be performed by the VR application and (b) the design of the test data using structural test criteria—applied in the specification—that aims at checking whether those functions are really performed by the software. The functions that the software should include are defined in their specifications. Thus, a well-designed specification that caters user requirements is essential for this type of test [31, 32].

We also present a tool named virtual requirement specification and testing (ViReST) here, which supports the automation of functional testing of VR applications that employ scene graph (SG). The ViReST tool allows the automated generation of test data using structural test criteria from the requirements written in a controlled natural language (CNL), called behavior language requirements specification (BeLaRS).

The main contributions are to provide an effective approach to requirements specification as well as to generate test cases for SG-based VR applications and a user-friendly tool for testing such applications. More specifically, this paper

- (i) sets out an approach and a tool for requirement specification as well as for testing VR applications;
- (ii) describes in detail a semiformal language for a more concise specification of requirements;
- (iii) allows the automatic generation of test requirements and provides test data using structural test criteria from the requirements that are specified using the semiformal language; and
- (iv) makes an evaluation of the effectiveness of test cases generated by the proposed approach.

It is important to emphasize that the usability tests performed in VR application are centered on the user. Thus, the possible flaws detected are directly related to the user perception. Regarding the functional testing, the main effort employed is during requirements specification, since these requirements are used as input to this activity (functional testing). In the context of the VR-ReST approach, the effort employed is not different and is directly related to the understanding of the BeLaRS language rules allowing the requirements specification in a standard that is in agreement with the proposed grammar in the language.

In Section 2, we provide an overview of some of the underlying concepts of this work. In Section 3, we outline the VR-ReST approach to generate test cases for VR systems and describe each of its phases. Section 4 provides more information about the ViReST tool, the test requirements, and the test cases generated by it. Two experiments are described in Section 5 to evaluate the approach and to interpret some of the features in the tool. Section 6 discusses the results of the approach, and Section 7 addresses related works. Finally, the conclusions and topics for further investigation are discussed in Section 8.

2 | BACKGROUND

This section presents the main concepts about VR particularities that should be considered during development and test of VR applications based on scene graphs. We also introduce four test criteria based on structural testing previously defined for SG-based VR applications testing.

2.1 | Virtual reality

VR can be defined as a highly advanced form of interaction between the user and the computer in real time in a 3D environment using multisensory devices [21]. The VE of VR applications allows users to interact by using a variety of devices, from simple ones such as keyboard, mouse, and video monitor to more complex devices such as datagloves, head-mounted displays (HMD), and haptic devices [21, 33, 34].

VR allows users to modify and to interact with 3D scenes involving virtual objects, which represent objects in the real or in an imaginary world. VR applications can provide different degrees of immersion, with different senses involved, such as vision, touch, and hearing, depending on the computational resources and equipment used [13].

According to Vince [35] and Kirner [14], the user can explore and even modify the VE, which is possible by using techniques involving navigation, interaction, and immersion. Kirner [14] states that *immersion* can be mental or physical. Mental immersion occurs when 3D virtual objects are only visualized in an output device without users experiencing the sensation of being within the VE. Physical immersion takes place when users employ devices, such as HMD or other stereoscopic devices, to try the sensation of being within the VE. *Navigation* is the ability of moving in the environment. *Interaction* offers an opportunity to interact with virtual objects and manipulate them as if they were real objects.

The high degree of complexity of the scenes in VEs has led to the need to define appropriate structures to represent them. An attempt was hence made to reduce these difficulties by employing a data structure, named SG, to specify complex scenes using a hierarchy of objects and attributes.

2.1.1 | Scene graph

A factor that intensifies the challenge of developing VR programs is that these programs have peculiar requirements. They are highly complex, requiring a rigorous specification of requirements. Due to the complexity of VEs in VR programs, most of the graphic libraries use the concept of SG to represent the VEs. SG organizes objects and their characteristics, such as behaviors and appearances, in the VE.

SG is a class-organized data structure capable of specifying complex scenes through a hierarchy of objects and attributes [36]. It is composed of nodes that represent objects, their characteristics, and behaviors. Each of these characteristics are related to the VE descriptions and interactions and, so, should be embedded in the requirements specification. Figure 1 presents the structure of a SG considering the different types of nodes, described as follow:

- *Root node* is the first node of the SG and represents the virtual universe. All the other nodes are either directly or indirectly linked to it.
- *Locale* defines an area with size and location in the virtual universe. The SG is composed of trees originating from a Locale, and there is only one path from the root of a tree to each of the leaf nodes.
- *Group node* is responsible for organizing the data, behaviors, control the rendering order, and changes the position, orientation and size of objects visible in the VE. It is composed of BranchGroup (BG) nodes and TransformGroup (TG) nodes. One or more geometric transformations (rotation, translation, or scale) may occur in a particular object of the VE. The transformations are directly related to the hierarchy of objects because if a grouping node is transformed, the leaf nodes will inherit their transformations. Behavior can be classified into two categories: deterministic and nondeterministic. Deterministic behaviors are all those whose states are defined as a function of time. This behavior allows the user to navigate back and forth in time, knowing exactly what the state of each entity is at any given time. In contrast, the nondeterministic behavior is unpredictable since it is the reflection of the user actions that do not follow a defined pattern [37].
- *Leaf node* represents an object of a graphic scene. This node contains the properties (geometry and appearance) of objects. Geometry represents the shape of the object that has to be processed. The geometrical description can be classified as either static or dynamic. In a static description, the shape of an object does not change, while in the dynamic description, the shape can be changed. Appearance influences the quality of the final image. The appearance can be related to color, material, texture, brightness, light, shadow, and the reflection of objects.

Figure 2 illustrates an example of a scene and its respective SG, from an application that aims to simulate a biopsy exam, generated by the ViMeT framework [15]. The scene represented in Figure 2A is composed by two virtual objects: a syringe and a breast. The scene has objects in blue and red colors in order to allow the view in stereoscopic mode, by using anaglyph glasses. Each property of the virtual objects such as shape, color, and behavior, presented in Figure 2A, is associated to a respective node in SG, as can be seen in Figure 2B. The mapping of these properties to the node of the SG is detailed in Table 1.

Figure 2B is composed of a BG3 BranchGroup node that represents the syringe. The transformations on the syringe can be represented by the TG2, TG4, TG41, TG42, and TG43 TransformGroup nodes, and its properties are represented by the *shape3D411* and *shape3D421* leaf nodes. The breast is represented by the BG4 BranchGroup node, its transformations are represented by the TG3, TG5, TG51, TG52, and TG53 TransformGroup nodes, and its properties are represented by the *shape3D511* and *shape3D521* leaf nodes. Figure 2 is also composed of a root node (VU1) that represents the entire VE. In the VE, BG3 involves the user view and BG2 is related to the VE's lighting. BG2 is composed by

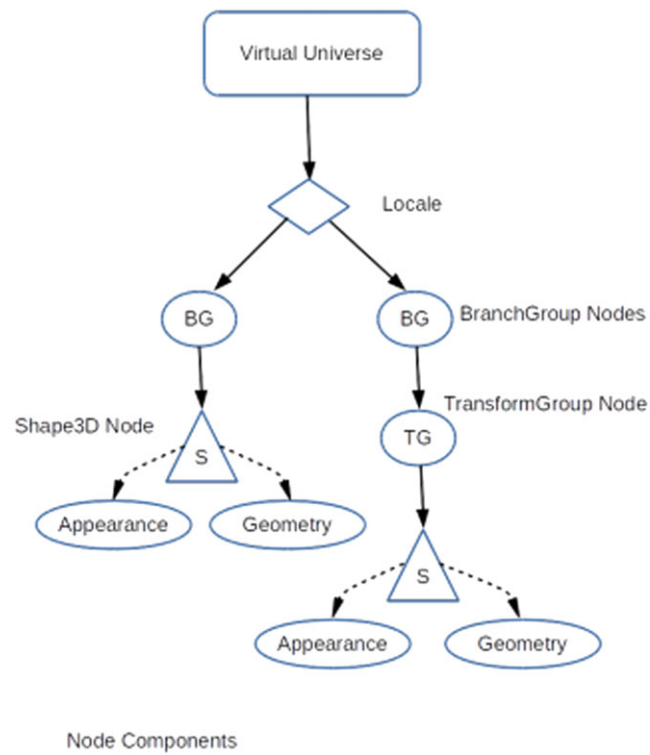


FIGURE 1 The scene graph structures of a virtual environment is composed of three types of nodes: root node, group node (BranchGroup [BG] and TransformGroup [TG]), and leaf node [36]

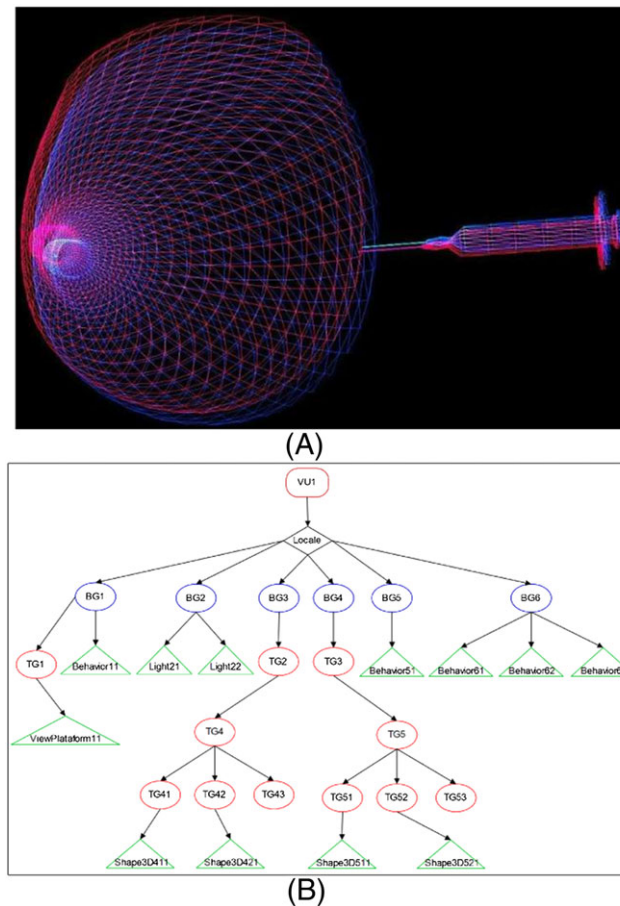


FIGURE 2 Example of a scene graph. A, VE to simulate breast biopsy, containing two virtual objects (breast and syringe). B, Respective scene graph of the VE presented in Figure 2A. BG, BranchGroup; TG, TransformGroup; VE, virtual environment

TABLE 1 Mapping of objects presented in Figure 2A into SG presented in Figure 2B

Objects of Figure 2A	Types of Nodes	Nodes of Figure 2B	Description
Syringe	BG	BG3	This node represents the syringe in the VE.
	TG	TG2, TG4, TG41, TG42, TG43	These nodes represent the transformations of the syringe.
	Leaf nodes	shape3D411, shape3D421	These nodes represent the geometry of the syringe.
Breast	BG	BG4	This node represents the breast in the VE.
	TG	TG3, TG5, TG51, TG52, TG53,	These nodes represent the transformations of the breast.
	Leaf nodes	shape3D511, shape3D521	These nodes represent the geometry of the breast.
Universe	Root	VU1	This node represents the entire VE.

Abbreviations: BG, BranchGroup; SG, scene graph; TG, TransformGroup; VE, virtual environment.

two leaf nodes (*Light21* and *Light22*). User-defined actions performed in the VE are represented by five leaf nodes (*Behavior11*, *Behavior51*, *Behavior61*, *Behavior62*, and *Behavior63*).

Within this context, it is possible to make use of test criteria that employ this model to find out the test requirements. An analogy can be drawn with the structural criterion, since by using SG, it is possible to select structures that can be employed during the testing activity.

2.2 | Test criteria

Test criteria are rules to assist test case sets generation and to evaluate the suitability of this set, by assessing the efficiency of the test [38]. A test criterion provides the guidelines for selecting a subset of test cases from all possible inputs. The goal is to select those that can expose the faults present in the software. In this context, the quality of the test set is associated with the test criterion used, since it defines which test requirements must be exercised.

A structural testing criterion is, generally, based on a control flow graph (CFG). A CFG is a directed graph generated from source code or, in some cases, other artifacts, such as use cases. In a CFG, each node represents a basic block of statements, and each edge represents the control flow between the basic blocks. A special node in the CFG, called initial or entry node, represents the entry point to the code represented by the CFG. A CFG can have final nodes, which represents the exit points of such code. A complete path in a CFG is a *pth* from the initial to a final node. A complete test path represents the execution of a test case on a CFG.

The structural testing criteria use CFG as a basis to define a set of test requirements (TRs) that must be fulfilled by the test set. The requirements produced by a criterion are technically predicates that can have either a value that is true (the requirement has been met) or false (the requirement has not been met). A typical test requirement is met by visiting a particular node or edge or by touring a particular path. Four criteria are used in this paper: node coverage (NC), edge coverage (EC), edge-pair coverage (EPC), and prime path coverage (PPC).

2.2.1 | Node coverage (NC)

NC is the simplest criterion and requires that all the nodes in the CFG be reached at least once. In other words, a test set T satisfies the node coverage on a CFG if, and only if, for every syntactically reachable node n of the CFG, there is some path p in the set of paths executed by T , such that p reaches n [2].

2.2.2 | Edge coverage (EC)

EC requires that all edges in the CFG be covered at least once, ie, a test set T satisfies the edge coverage on the CFG if, and only if, for every syntactically reachable edge e of the CFG, there is some path p in the set of paths executed by T , such that p reaches e [2]. The TRs derived from EC contain each reachable path that has a length of up to one, ie, the path include edges and also single nodes [39].

2.2.3 | Edge-Pair Coverage (EPC)

For EPC, TRs should contain each reachable path of length up to two. The idea is to exercise every pair of adjacent edges. The qualification “up to two” is included to ensure that EPC subsumes EC and NC in graphs that have only paths of size one or only one node [39].

2.2.4 | Prime path coverage (PPC)

The Prime Path Coverage (PPC) test criterion was defined by Ammann and Offutt [39] to ensure that there was a high coverage of loops without requiring an infinite number of paths. A simple path has no repeated nodes, although the first and the last nodes can be the same. A simple path from nodes n_i to n_j is a prime path if it is simple and does not appear to be a proper subpath of any other simple path [39].

PPC requires that all the prime paths in the CFG be reached. PPC includes two special cases that address loops in the paths. Both cases involve a treatment of loops with “round trips.” A round trip path is a prime path of nonzero length that starts and ends at the same node. The first case called simple round trip coverage (SRTC) requires, at least, one round-trip path for each reachable node in the CFG that begins and ends a

round-trip path. The second-case called complete round trip coverage (CRTC) must contain all the round-trip paths for each reachable node in the CFG.

3 | VR-REST APPROACH

In this section, we outline the details of our approach named VR-ReST aiming at supporting the requirements specification and test data generation for VR applications. VR-ReST uses high-level specifications (based on the use of case description) and domain concepts (terminology and concepts of the SG) to express the requirements of VR applications. VR-ReST also uses the four test criteria, described in Section 2.2, to derive the test requirements (TRs) and to generate test data from the specified requirements.

VR-ReST aims to guarantee a more rigorous specification based on SG concepts, so that some characteristics of VE, objects and the actions performed by multiple devices can be described in the specification of requirements and consequently tested. It must be highlighted that we do not use SG graph, but a graph based on control flow graph (CFG) named requirements flow graph (RFG). We use only the SG concepts in the requirements specification that is converted into RFG.

As shown in Figure 3, the VR-ReST approach consists of three sequential modules, namely, (1) requirements description, (2) requirements mapping, and (3) test generation.

The first module of the approach (step a in Figure 3) aims at collecting general information about applications and users as well as at identifying requirements. The output of this module is a set of requirements written in natural language.

The second module (steps b and c in Figure 3) aims to generate the RFG from a formal specification obtained using BeLaRS. BeLaRS is a semiformal language that will be presented in Section 3.2.

The last module (steps d to f in Figure 3) aims to execute functional testing. To go from the RFG to the TRs, a given test criterion is applied to the RFG. Once the TRs are derived, the test data are generated and applied to verify which TRs have been covered. The second and the third modules of our approach are supported by the ViReST tool, which will be explained in Section 4.

3.1 | Module 1: requirements description

The ViReS model was designed for guiding the requirements specification of VR applications and to obtain essential information that will be drawn on the mapping of requirements of our approach. The model provides the means to support the requirements specification related to characteristics and interactions of VE.

The proposed ViReS model was developed based on a mining of the best characteristics of some methods proposed in previous studies [14, 22, 23, 24, 25, 26, 27]; strategies of usability requirements specification for augmented reality systems proposed by Nakamoto et al [40]; definition of behaviors [41]; elements and technology; and phases of the traditional requirements engineering employed in software engineering (SE).

The ViReS model consists of four interrelated phases, as can be seen in Figure 4. It is composed by main and alternative flows. The main flow represents the basic flow that covers what normally should happen and the alternative flow consists to specify new or to modify existing properties of objects, VE, and behaviors that were not described in the main flow. Each phase is described as below:

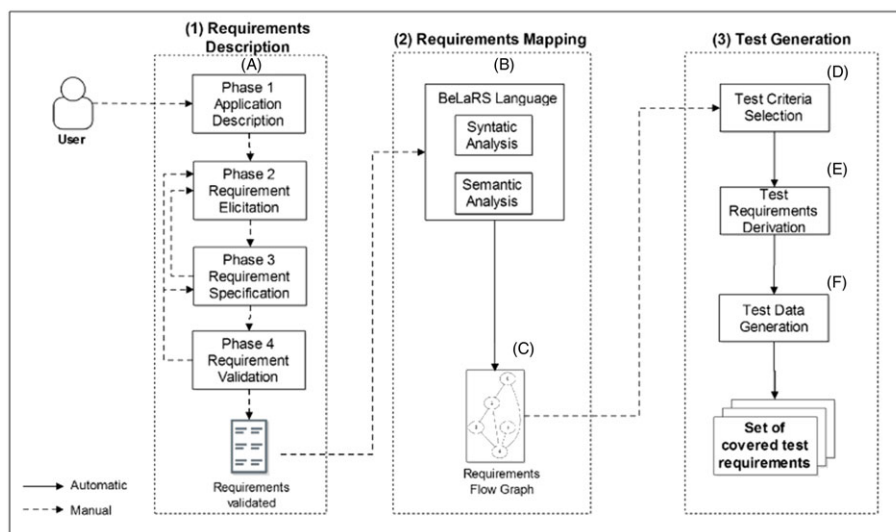


FIGURE 3 The VR-ReST approach process. Module 1: Requirements are specified through ViReS model; module 2: The requirements specified are used as input in the BeLaRS language to obtain a formal specification that is converted into RFG; module 3: From the RFG generated the TRs and test data are derived and applied in TRs to verify their coverage. BeLaRS, behavior language requirements specification; RFG, requirements flow graph; TR, test requirement; ViReS, virtual requirement specification; VR-ReST, virtual reality—requirements specification and testing

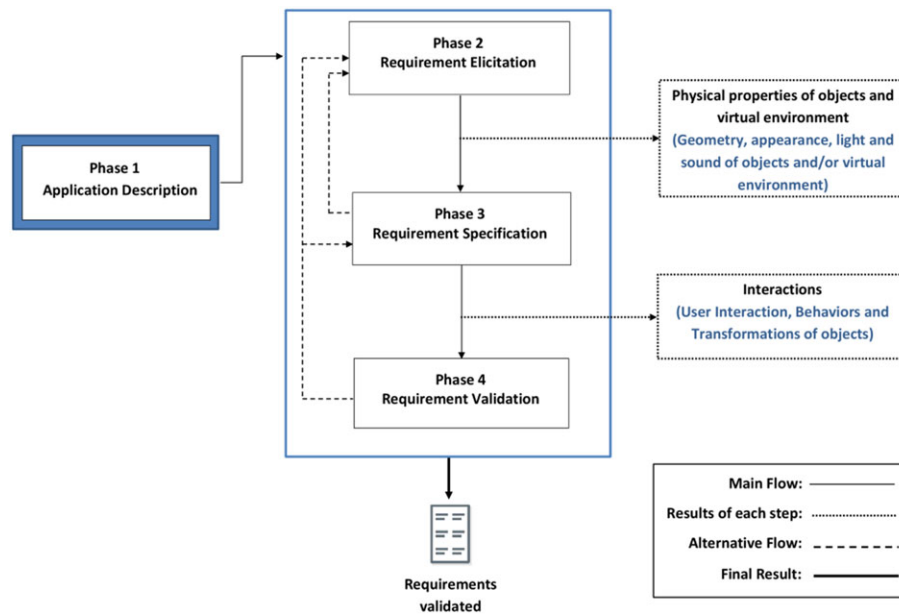


FIGURE 4 ViReS model is a guide to requirements specification of VR applications. This figure is a close look of Figure 3. Phase 1: The application description consists of an overview of application and users. Phase 2: Requirement elicitation is related to the properties of objects and VE existing in the VR application. Phase 3: Requirement specification consists of the description of the interaction between the user and VR application. Phase 4: The requirements specified are validated to verify if they conform with the needs of the intended application. VE, virtual environment; ViReS, virtual requirement specification; VR, virtual reality

3.1.1 | Phase 1: application description

This phase is part of the main flow and refers to a general application description, including the following: a definition of the problem domain, setting out goals, a view of those involved and the target group. Furthermore, the identification of the users' profiles as well as their level of knowledge and/or experience with VR technology is necessary to assess their skills and difficulties in the problem domain. The results of this phase will compose the documentation regarding the application to be developed.

3.1.2 | Phase 2: requirement elicitation

This phase is part of the main flow and aims at picking up and gathering domain knowledge, ie, determining the physical properties of objects and characteristics of the VE, as well as all devices that will be used and how many scenes will compose the VE. The output of this phase consists of the elements of the elicitation that are related to the SG structure, such as objects, the geometry of objects, the appearance of objects and VE, transformations and behaviors. These elements will be used as inputs and mapped by the BeLaRS language (Section 3.2) to describe the characteristics of the VE.

3.1.3 | Phase 3: requirement specification

Requirement specification is a comprehensive description of the essential behavior and intended environment for the software under development. This phase is part of the main flow and describes what the software is expected to perform. It also aims to specify the functional requirements that will compose the VE. The output of this phase consists of the requirements that represent all the information about the objects and the VE, as well as the types of transformation and behavior described in the previous phase. This phase has an alternative flow for phase 2. This flow occurs when some requirement needs a modification so that new requirements are specified. These requirements will also be mapped by the BeLaRS language to describe the user interactions with the VE as can be seen in Section 3.2.

3.1.4 | Phase 4: requirement validation

Requirement validation aims to show that the requirements conform with the needs of the intended application and thus ensure that the requirements are consistent, unambiguous, correct and complete. This phase has two alternative flows for both phases 2 and 3. The alternative flow for phase 2 allows modifying or adding information about objects and VE properties. The alternative flow for phase 3 allows modifying or adding user interactions, behaviors, and transformations of objects if they do not meet the needs of the application. These requirements are validated again. The output of this phase is a guideline to enable designers and developers to know if the specifications have been carried out and to record any unforeseen changes that occur during the development of the application. Therefore, all the validated requirements will be mapped by the BeLaRS language.

TABLE 2 Phases and activities of ViReS model based on distinct methods and concepts

ViReS Phases	Activities	Approaches/Concepts	References
Phase 1	Delimit and define the problem domain	Informing design of VE	[26]
	Identify the benefits of using VR technologies		
	Define target audience	Requirement specification strategy	[40]
	Identify the users profiles, their capabilities, and difficulties in the problem domain	Informing design of VE	[26]
	Identify the knowledge level and/or user experience with VR technology	Requirement specification strategy	[40]
	General description of the system, identifying the vision of those involved	Informing design of VE	[26]
		RE Process	[42]
Phase 2	Identify objects	CLEVR	[23]
		VRID	[25]
	Identify scenes	VR-WISE	[24]
	Identify geometry of objects	CLEVR	[23]
		VRID	[25]
	Identify appearance of objects	CLEVR	[23]
	Identify transformations of objects	VRID	[25]
		VR-WISE	[24]
	Define users	Design of VE	[43]
		Model for developing VE	[14]
		VRID	[25]
	Define type of VE: immersive/nonimmersive	RE Process for VE	[44]
		Design of VE	[23]
		RE Process for VE	[44]
	Define input/output technologies	VRID	[25]
		Model for developing VE	[14]
Phase 3	Select		
	Manipulate		
	Navigate	Behavior definition	[41]
	System control		
	Deform	Elements and technology	[41]
	Detect collision	VR-WISE	[24]
		Elements and technology	[41]
Phase 4	Define restrictions related to devices	Requirement specification strategy	[40]
	Define restrictions related to the rendering of images	Elements and technology	[41]
	Define restrictions related to system response to user commands		
	Review the requirements specified by checking the existence of incompleteness, ambiguity	RE process	[42]
	Validate the requirements specified in accordance with the usability criteria	RE process MAUVE criteria	[42] [27]

Abbreviations: CLEVR, concurrent and level by the level development of VR system; MAUVE, Multi-criteria Assessment of Usability for Virtual Environments; RE, requirement engineering; VE, virtual environment; ViReS, virtual requirement specification and testing; VR, virtual reality; VRID, virtual reality interface design; VR-WISE, A Conceptual Modelling Approach For Virtual Environments.

The requirements must be validated by a checklist. The questions in the checklist should point out the issues and possible risks that may appear in the requirements specification guideline.

The activities composing the phases aforementioned consider aspects of scenes commonly used in SGs, such as the description of the VE (VED) and the description of the interactions (IND). Table 2 presents a description of the activities of each ViReS model's phase. This table shows the ViReS model phases (first column), the activities performed in each phase (second column), the approaches/concepts used to define the activities (third column), and the references that underlie such approaches and concepts (fourth column).

3.2 | Module 2: requirements mapping

In order to support the mapping of the requirements specified in Section 3.1, we defined and developed a language named behavior language requirements specification (BeLaRS).

BeLaRS is a semiformal language that enables a formal specification of requirements to assist the functional testing in the VR domain and that uses Chomsky theory [45], parts of speech (POS) [46], and case frame (CF) and uses case descriptions [47] and SG concepts to ensure a formal requirements specification to support the functional testing of VR application.

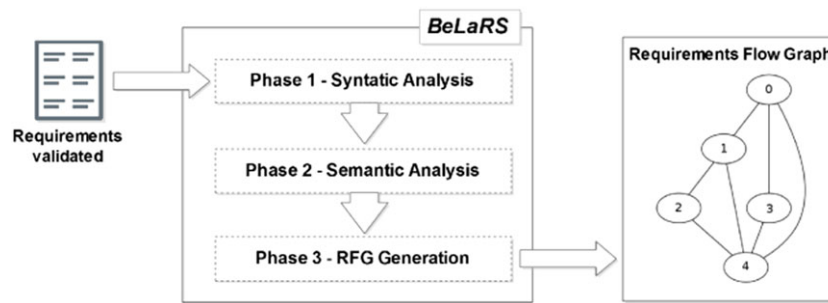


FIGURE 5 Phases of the BeLaRS language, which enables the requirements mapping (second module of the VR-ReST approach shown in Figure 3). Phase 1: syntactic analysis of validated requirements. Phase 2: mapping the syntax trees into a semantic representation. Phase 3: turns the semantic representation into RFG, which will be used to produce the TRs and test data. BeLaRS, behavior language requirements specification; RFG, requirements flow graph; TR, test requirement; VR-ReST, virtual reality—requirements specification and testing

The requirements specified using BeLaRS describe functional requirements concerning a CNL [48] and use case descriptions and SG concepts employed to describe VR applications, to allow a semiformal specification of requirements that is free from any vagueness and ambiguity and closer to the user's level of understanding.

We use the CF because it can be considered a specification of interconnected sequences of actions to perform between actors and system. The SG terminologies and concepts employed aim to describe the features of a VE, as well as the interactions between users and applications.

The Requirements Mapping using the BeLaRS language comprises three phases, as illustrated in Figure 5, and detailed below: (1) syntactic analysis, (2) semantic analysis, and (3) requirements flow graph (RFG) generation.

3.2.1 | Phase 1: syntactic analysis

The first phase is responsible for verifying whether the system requirements are written according to BeLaRS. For each valid requirement (input), the BeLaRS-Parser generates the corresponding syntax tree, which is used as input to the semantic analysis phase. Our language is defined by a phrase structure context-free grammar and the lexicon containing the SG domain vocabulary.

Concerning the BeLaRS lexicon, its entries are classified into categories, also known as parts of speech (POS) [46]. In this work, we consider the following categories:

- Determiners (DETER), words that determine (limit) the meaning of a noun (for example, a number, an article, or a personal pronoun) [49];
- Nouns (NSING for singular and NPLU for plural) that can represent objects (for example, ball and car), actor, or entity;
- Verbs with inflections (for example, VTOBE-PRE—for the verb to be in the present form);
- Prepositions (PREP).

In order to simplify the BeLaRS grammar rules, we created four special categories:

- NUMBER for digits (1, 2, 3, 4, 5, 6, 7, 8, 9);
- SYMBOL for special symbols (eg, “:”, “,”, “;”, “(”, “)”, “+”, “-”);
- PARAMETER for a combination between SYMBOL and DIGIT (eg, (3.1), 3.1.1);
- CONTROL for control structures (if, else, while, for, and case).

Furthermore, we utilized SG concepts and use case description as special entries to identify keywords. Table 3 presents the entries used in the grammar definition.

We adopt the theory of syntactic structures, devised by Chomsky [45], which assumes that sentences can be divided into a noun phrase (NP) and a verb phrase (VP). We employed Chomsky theory in the BeLaRS language because it uses the sentence structure (subject, verb and predicate). This theory facilitates the requirements specification since all users have knowledge of this structure because it is close to the natural language and allows to generate a grammar that fulfills this structure.

Hence, the BeLaRS grammar consists of sentences structured according to some rules, which define the syntactic structures accepted by our parser. Our grammar was defined as a context free grammar (CFG), represented in the extended Backus-Naur form (EBNF) notation (Figure 6).

The grammar start symbol is *Scenario*, which consists of an *EnvironmentReq* and an *InteractionReq*. Thus, the requirements represent VE descriptions and interactions between users and system. Figure 7 illustrates an example of a requirement specification and their respective syntactic tree generated using the grammar presented in Figure 6.

The grammatical system proposed can represent requirements written with several different formations for VR applications based on SG concepts. However, this grammatical system can assist the needs of different areas of application. We have successfully applied the BeLaRS to different application fields, such as medical training and games, which were considered herein.

TABLE 3 Entries and keywords of the BeLaRS language

Domain	Entries	Keywords
Use case	Main path	Main path
	Alternative path	Alternative path
	ACTOR	User, user
	ENTITY	System, application, VE
Scene graph	GEOMETRY	Point, line, vector, vertex, polygon, triangle, sphere, cone, cube, cylinder, box, curves, surfaces
	POSITION	Upper right corner, upper left corner, lower right corner, lower left corner, inside left, inside right, center, right side, left side, underside, upper, top center, bottom center
	APPEARANCE	COLOR, transparency, texture, brightness
	COLOR	White, black, red, blue, yellow, green, orange, purple, pink, gray, silver, brown, gold, lime, violet, magenta
	LIGHT	Ambient, directional, reflective, punctual
	SOUND	Background, point, cone, soundscape
	VTRANSFORMATION	Rotates, translates, scales
	VBEHAVIOR	Selects, clicks, inserts, moves, moves up, moves down, moves right, moves left, moves out, moves into, moves into, deforms, detects collision, navigates, manipulates, releases, opens, closes, shows, presents, changes, makes, does, picks, drops
	DEVICE	Gloves, head, mounted displays, HMD, joysticks, haptics, mouse, keyboard
Control structure	CONTROL	If, else, for, while, case

Abbreviations: BeLaRS, behavior language requirements specification; VE, virtual environment.

SCENARIO	→	EnvironmentReq InteractionReq
EnvironmentReq	→	NounPhrase VerbPhraseDesc
NounPhrase	→	DET? NUMBER? Noun DET? Noun NUMBER? Noun
Noun	→	NSING NPLUR
VerbPhraseDesc	→	Verb Verb Adj Verb NounPhrase+ (Verb (Adj NounPhrase)+)
Verb	→	VTOBE_PRE VAUX_PRE VAUX_PP
Adj	→	(GEOMETRY POSITION APPEARANCE LIGHT SOUND) Parameter?
Parameter	→	(NUMBER FLOAT) COMMA ((NUMBER FLOAT)+) [(NUMBER+ DOT NUMBER+) (NUMBER+ DOT NUMBER+ DOT NUMBER+)]
InteractionReq	→	MainPath AlternativePath?
MainPath	→	NounPhrase VerbPhraseAction VerbPhraseComplement Parameter?
VerbPhraseAction	→	(VBEHAVIOR VTRANSFORMATION) NounPhrase?
VerbPhraseComplement	→	NounPhrase? PREP DEVICE? NounPhrase PREP? (Adj+ NounPhrase) ((PREP Adj? NounPhrase)?) MainPath
AlternativePath	→	Parameter CONTROL NOT? VerbPhraseAction VerbPhraseComplement VerbPhraseCondition
VerbPhraseCondition	→	Parameter NounPhrase VerbPhraseAction (Adj NounPhrase)?

FIGURE 6 Behavior language requirements specification (BeLaRS)—a grammar for requirements specification. The grammar is composed of words in lower case that represent nonterminal symbols and uppercase that denote terminal symbols that correspond to lexical categories

3.2.2 | Phase 2: semantic analysis

The second processing phase receives the generated syntactic tree as input and provides the semantic representation. In this study, the case grammar theory is adopted [50] to represent the meaning of sentences. In this theory, a sentence is analyzed regarding the semantic (thematic roles, *Trs*) of each word/group of words in the sentence (eg, agent, patient, location, object, and instrument).

The verbs associated to *Trs* are aggregated into a CF. A CF can be viewed as a structure representing a verb's thematic role, to be filled by sentence elements. This means that the *Trs* in a CF specifies the structural context of the verb. This study considers nine *Trs*, which are based on the nomenclature adopted by Allen [46] (Table 4).

From the *Trs* defined, it is possible to explain how the contents of each thematic role can be inferred from the syntax trees. Table 5 presents the 10 rules (R_n) for the *Trs*.

These rules need to be updated if new verbs are included when writing the requirements. However, this set of verbs and rules were sufficient to describe requirements for different domains and structures as is described in Section 6.

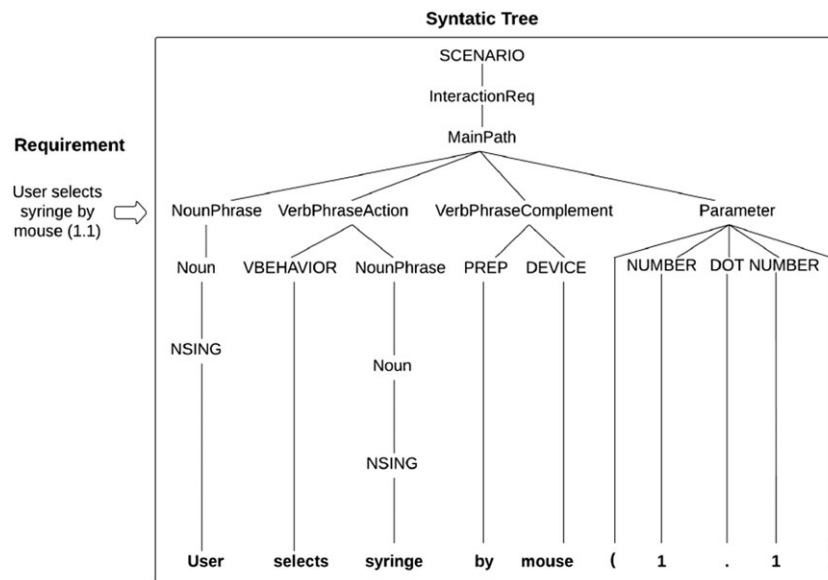


FIGURE 7 An example of a requirement specification and their respective syntactic tree generated using the behavior language requirements specification (BeLaRS) grammar

TABLE 4 Specific Trs used in the behavior language requirements specification (BeLaRS) grammar

	Trs	Descriptions
Action statement	Action (ACT)	The action performed if the conditions are satisfied and to describe features about the entity
	Agent (AGT)	Entity who performs the action
	Patient (PAT)	Entity who is affected by the action
	Instrument (INST)	The means by which an action is performed
	Location (LOC)	The place in which entity or something is situated or takes place
	Goal (GL)	The entity towards something moves
Condition clause	Condition action (CACT)	The action that concerns each condition
	Condition patient (CPAT)	The entity related to each condition
	Condition modifier (CMOD)	A modifier related to the condition

3.2.3 | Phase 3: requirements flow graph generation

The requirements flow graph (RFG) generator receives the validated requirements by using BeLaRS in semantic analysis phase and generates the corresponding RFG. This graph is used to derive TRs and to generate test data. Our work refers to the Prefuse library [51] for automatically generating RFG from the requirements specified.

The RFG generation process is composed of two stages: (a) combination of each requirement for the graph nodes and (b) RFG generation. Figure 8 presents an example of a RFG generation.

In the combination of each requirement for the graph nodes (first stage of the process of RFG generation), the requirements are converted into a RFG as can be seen in Figure 8. Each specified requirement corresponds to a node that will compose the RFG. In the main path, each requirement is linked to a node. In the alternative path, in the case of a given requirement that represents a condition (for instance, requirement 1.1), one or more requirements from this condition are linked to a single node corresponding to this condition. For instance, requirement 1.1 has a condition; hence, its downward requirement (1.1.1) is linked to the node that represents requirement 1.1.

3.3 | Module 3: test generation

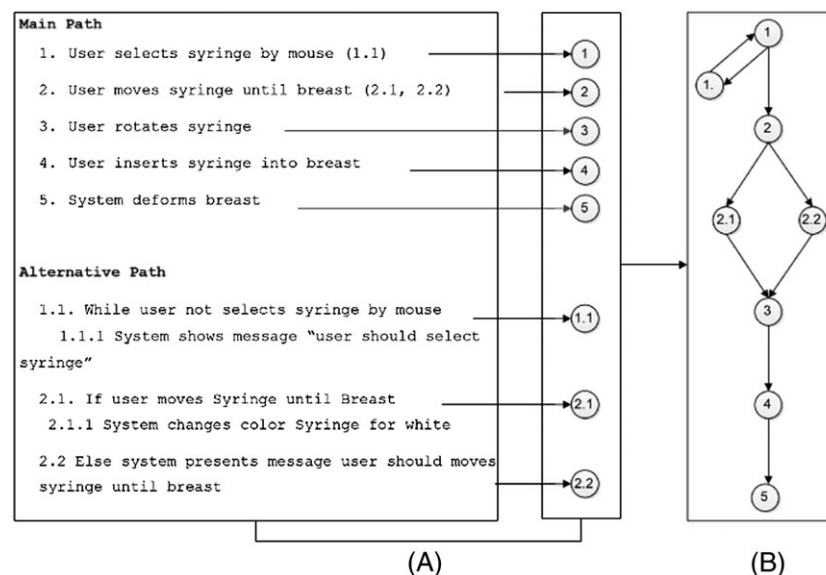
After deriving the RFG presented in the requirements mapping (module 2) of the Figure 3, we perform the functional testing. First, we obtain all the TRs by following each criterion (NC, EC, EPC, and PPC), presented in Section 2.2, and then we generate a test data set that traverses the RFG, moving from the initial node to the final node.

While traversing the RFG, we used the depth first search (DFS) [52] optimization technique. As its name implies, DFS traverses “deeper” into the graph whenever possible. In DFS, the edges (e) are explored away from the most recently discovered vertex (v) that still has an unexplored edge. This process continues until we discover all the vertices that are reachable from the vertex of origin. If any undiscovered vertex remains, then one of them is selected as a new origin, and the search is repeated from that. The whole process is repeated until all the vertices have been discovered.

The test data generation strategy is employed from RFG, as can be seen in the example presented in Figure 8. This strategy aims to derive the TRs and generate test data, as can be seen in Figure 9.

TABLE 5 Rules and examples to explain how the content of each thematic role is used in the behavior language requirements specification (BeLaRS)

Rules	Description	Example
R ₁ . ACT	Comprises the terminals of: {(i)} Concerning the verbs {VTOBE_PRE}, {VAUX_PRE}, {VAUX_PP} to describe features of VE that shall be used only in {VerbPhraseDesc}; {(ii)} Concerning the verbs {VTRANSFORMATION} and {VBEHAVIOR} to represent the actions performed and shall be used in {VerbPhraseAction}, {VerbPhraseComplement} and {VerbPhraseCondition}.	(i) VE has Sun, Moon. (ii) User selects Sun by mouse.
R ₂ . AGT	Comprises the terminals of the {NounPhrase} that can be an Actor, an Entity or a noun.	User scales Sun.
R ₃ . PAT	Comprises the terminals of the {NounPhrase} that is present in the {VerbPhraseCondition}.	If user not moves Sun until Moon. System presents error message.
R ₄ . INS	Comprises the terminals of the DEVICE that is present in the {VerbPhraseComplement}.	User rotates Sun by mouse.
R ₅ . LOC	Comprises the terminal of the POSITION that is present in the {VerbPhraseDesc}, {VerbPhraseComplement} and {VerbPhraseCondition}.	Sun is upper right corner.
R ₆ . GL	Concerning the verbs moves, moves up, moves down, moves right, moves left, moves out, moves into, the structure is: moves X to Y or moves X until Y.	User moves up Sun to Moon.
R ₇ . CACT	Comprises the terminals of {VerbPhraseCondition}.	If user not selects Sun.
R ₈ . CPAT	Comprises the terminals of the {NounPhras}e that is present in the {VerbPhraseCondition}.	While user not moves Syringe
R ₉ .	Concerning the verb to change, the structure is: changes X to Y.	System changes Moon color to white
R ₁₀ . CMOD	Comprises the terminals of {VerbPhraseCondition}.	If user rotates Sun. System deforms Sun.

**FIGURE 8** Example of a requirements flow graph (RFG) generation. A, In the first stage, the requirements specified in the main and alternative path correspond to a node. B, In the second stage, each node identified composes the RFG. The RFG is generated according to the control structures described in the requirement specification

TR1:	{ 1 }
TR2:	{ 1.1 }
TR3:	{ 2 }
TR4:	{ 2.1 }
TR5:	{ 2.2 }
TR6:	{ 3 }
TR7:	{ 4 }
TR8:	{ 5 }

(A) Test Requirements

T1:	{1, 1.1, 2, 2.1, 3, 4, 5}
T2:	{1, 2, 2.2, 3, 4, 5}

(B) Test Data

FIGURE 9 Example of the A, test requirements (TRs) and B, test data generated according to the node coverage (NC) testing criterion from the requirements flow graph (RFG) generation presented in the Figure 8

In Figure 9, the TRs (TR1, ..., TR8) are derived while the RFG is traversed according to NC criterion. From this, the goal is to generate a set of test data (T1 and T2) guided by the testing criterion that is able to cover the TRs derived. If a given test data is not able to cover the TR, then new test data should be generated until all TRs have been covered. If that is not possible, for a given test criterion, the user can manually add new test cases.

4 | ViReST TOOL

For making the use of the VR-ReST approach feasible, we implemented a tool named ViReST. This tool allows the requirements to be specified in a standardized way and to automatically generate and execute test data by employing the concepts outlined in the preceding sections. ViReST was entirely built using the Java programming language, and it is composed of two parts: requirement component and test component, described as follows.

4.1 | Requirements component

The *Requirements Component* covers the specifications of all the requirements of the user-system interactions and object features needed to carry out the functional testing. The tool aims to cover the activities related to the requirements description, requirements mapping, and test generation modules described in Sections 3.1, 3.2, and 3.3, respectively.

In the *Requirements Component*, development was also used the another tool for language recognition (ANTLR) that consists of a parser generator used to read, process, execute, or translate structured text or binary files [53].

The specification in the *Requirements Component* includes the *name*, together with a short *description of the application* and *general description of the scenario*, *VED* and *IND* descriptions. An example of a requirements specification using BeLaRS in the ViReST tool is presented in Figure 10.

In the requirement menu (region A in Figure 10), the specified requirements can be stored by using the *save option* and imported using *load option*. Furthermore, still in the part A, the specifications are stored in an *xml* file and can be changed in two ways: (1) by using the ViReST or (2) directly on the *xml* file. Also, the user can get the specification performed in a textual form using the *show as text* option.

Each scenario of a given application and its respective requirements can be either added by *Add* button or excluded by the *Del* button (region B in Figure 10).

The requirements related to characteristics of VE and objects are specified in the VED (region C in Figure 10), and the requirements about the user-system interactions are specified in the IND (region D and E in Figure 10).

In the IND, more specifically in the basic flow (region D in Figure 10), the user can specify the interactions and indicate the existence of one or more alternative flows using the *Alt. Flow* option (region E in Figure 10).

As long as the requirements are specified in the VED and IND, the user should run the specification using *Run* button (region F in Figure 10), to check whether or not it complies with the rules of BeLaRS. If the specification is not correct, a list of possible errors that need to be corrected is shown in the *log* space (region G in Figure 10).

Note that, in this component, the user can perform the requirements specification for characteristics of the VE, behaviors, and transformations of objects. From the specification using BeLaRS, the user can validate the specification by executing it. After that, the errors detected are presented in the *log* space by the number of lines and the type of error. For instance, "Parser error: line 27:5: extraneous input user' expecting Else, Case, While, If, For, DIGIT." Therefore, as an output, this component provides complete, concise, and ambiguity-free requirements specification that will be used to carry out the tests in the next component.

ViReST

Requirements (A)

Application: Biopsy Test
Description: biopsy test with code

Scenarios (B)

Clear Scen. Add Del

Num	Description
1	Scenario 1

Virtual Environment Descriptions (C)

Add Del

Description

Virtual environment has Syringe, Breast
Virtual environment has punctual light, background sound
Syringe is cube (1, 1, 1)
Breast is box (1.0, 2.5, 1.0)
Syringe is blue
Breast is magenta
Syringe is upper right corner

Interactions Descriptions

Main Path (D)

Add Del

Num	Description	Alt. Path
1	User selects Syringe by mouse	1.1
2	User moves Syringe until Breast	2.1, 2.2
3	User rotates Syringe	
4	User inserts Syringe in Breast	4.1, 4.2
5	System shows Breast	

Alternative Path (E)

Add Del

Num	Description
1.1	While user not selects Syringe
1.1.1	System shows message User should select Syringe
2.1	If user moves Syringe until Breast
2.1.1	System changes color Syringe for while
2.2	Else System presents message user should mov...
4.1	If user inserts Syringe into Breast
4.1.1	System deforms Breast
4.2	Else System presents message user should insert...

Log (G)

13-09-2018 17:35:28 - Build success

Run Gen. Graph

Clear Log (F)

FIGURE 10 An example of a requirements specification in the *Requirements Component* using the virtual requirement specification and testing (ViReST) tool. In region A, it is possible to describe the application, save, and import the specified requirements. In region B, each scenario of the application can be added or excluded. In region C, the characteristics of the virtual environment (VE) and objects are specified. In regions D and E, the user-system interactions are specified. In the region F, the requirements specified are executed. In the region G, a list of possible errors that need to be corrected is shown

4.2 | Test component

The test component is a way of carrying out functional testing. It uses the requirements specified by BeLaRS (Section 3.2) as input and converts them into a RFG, as presented in Section 3.3.

The testing requirements can be derived from the RFG by using the DFS technique presented in Section 3.3. Furthermore, the test data are generated for testing requirements in compliance with particular test criteria, such as NC, EC, EPC, and PPC (Section 2.2). The aim of this component is to provide a systematic method of employing test criteria to maximize the VR-ReST ability to detect faults.

The requirements specified in Figure 10, their test requirements, the test data generated, and the results of the test are illustrated in Figure 11.

From RFG (region A in Figure 11), the user can select which test criteria must be used to derive the test requirements and generate test data using the test criteria option (region B in Figure 11). Once the test criterion has been selected, the test requirements are automatically generated (region C in Figure 11).

After the generation of the test requirements, the user must decide how the test data will be generated, which can be done automatically by the *Generate* button or manually by *Add* button (region D in Figure 11). Also, the user can exclude any test data by using the *Delete* button. Then the user must execute the test data set that has been generated by using the *Run* button (region E in Figure 11).

The results of this test, ie, the test requirements covered or not covered by each test data, are shown in the *Log View* space (region F in Figure 11). These results can be stored by using the *Report* option (region G in Figure 11).

In this component, the user can run the functional testing automatically from the requirements specification performed in the requirements component. From the specification, the RFG is produced, and the user can choose which criteria will be applied to generate the test requirements. After that, the test requirements are automatically generated and presented to the user. The user also can choose if the test data will be generated automatically or manually. After the execution of these test data, the user can see the test requirements coverage and the test data used in the log view (for instance, "TR1 was covered by T1 and T2 using node coverage"). Therefore, as an output, this component provides the coverage of test requirements by test data and test criteria.

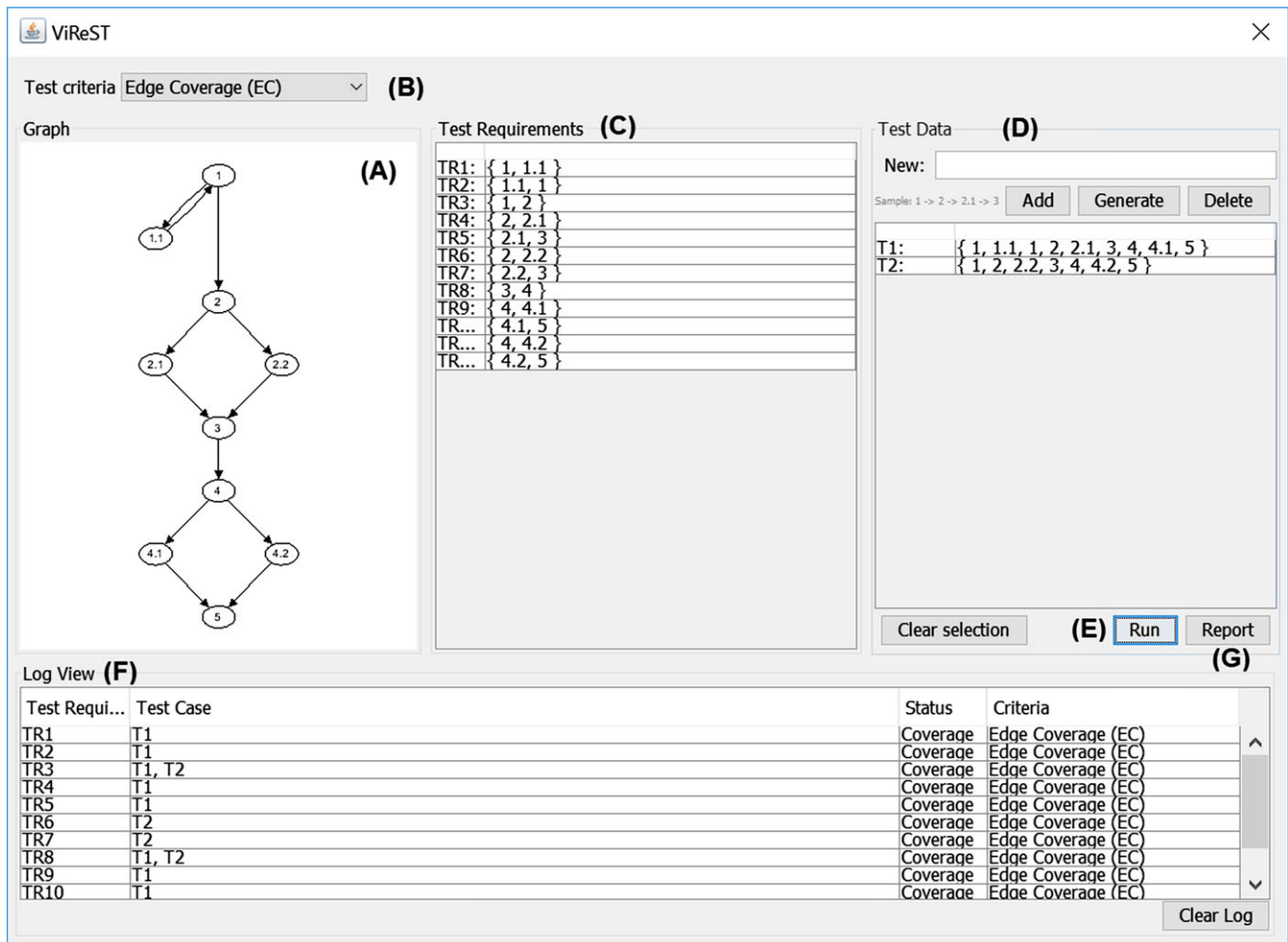


FIGURE 11 An example of test execution in the *Test Component* using the virtual requirement specification and testing (ViReST) tool. In region A, it is possible to view the requirements flow graph (RFG) generated from the specified requirements. In region B, a test criterion is selected. In the part C, the test requirements are automatically generated for selected criterion. In region D, test data are generated. In region E, the test data are executed. In region F, the test requirements covered or not covered by each test data are presented. In region G, the results can be stored in a report

5 | EXPERIMENTAL STUDY

Two empirical studies were conducted to investigate: (1) feasibility of the VR-ReST approach, more specifically the BeLaRS language effectiveness, and (2) effectiveness and applicability of the VR-ReST approach by the ViReST tool.

The first experiment consisted in an exploratory case study to evaluate if BeLaRS conforms to VR concepts and features and how easy is to use this language to specify the requirements. The second experiment was a controlled investigation to determine if the proposed approach is effective in performing functional testing on VR applications with the ViReST tool. In both studies, the guidelines recommended by Wholin et al [54] were used.

5.1 | Case study to evaluate the BeLaRS effectiveness

The aim of this case study was to evaluate how our proposed language is effective for requirements specification of VR applications. The study was conducted to find out whether there were any possible problems of conformity and usability in the BeLaRS.

Figure 12 shows a requirement specification using BeLaRS to describe the application that simulates a biopsy examination. The specification using BeLaRS is divided into two parts: the first (VED) consists in the description of objects and characteristics of the VE, and in the second (IND), the interactions user-system, highlighting the transformations and behaviors of objects, are addressed.

5.1.1 | Research design

The case study was conducted with 31 volunteers graduate, in Computer Science from three different universities (two in Brazil and one in Luxembourg). Three of them were expert in the use of VR applications, 17 had intermediate experience, 8 had little experience, and 3 had no

VED	IND
1. Virtual environment is black 2. Virtual environment has punctual light 3. Virtual environment has syringe and breast 4. Syringe is a cylinder 5. Syringe is blue 6. Syringe is on the right side 7. Breast is a cube 8. Breast is pink 9. Breast is on the left side	* Basic Flow: 1. User selects syringe by mouse (1.1) 2. User moves syringe until breast (2.1, 2.2) 3. User rotates the syringe 4. User inserts the syringe on the breast 5. System deforms breast * Alternative Flow: 1.1 While user not select syringe 1.1.1 System shows message "Select the syringe" 2.1 If user not moves syringe until breast 2.1.1 System shows message "Move syringe until breast" 2.2 Else System changes color syringe to red

FIGURE 12 An example of virtual requirements description (VED) and interaction description (IND) specification using behavior language requirements specification (BeLaRS)

TABLE 6 Conformity research questions and hypotheses

Research Questions	Hypothesis
C-RQ ₁ : Is there simplicity in the BeLaRS based on VR concepts?	C-H1₀: There is no simplicity in the BeLaRS because it does not use sufficient concepts and does not support users in undertaking their work. C-H1₁: There is simplicity in BeLaRS because it uses several VR concepts and supports users in undertaking their work.
C-RQ ₂ : Is there orthogonality in BeLaRS based on distinct VR concept?	C-H2₀: There is no orthogonality in BeLaRS because the keywords do not represent a different concept of VR. C-H2₁: There is orthogonality in BeLaRS because each keyword represent distinct concepts of VR.
C-RQ ₃ : Is there scalability in BeLaRS for large-scale requirements specification?	C-H3₀: There is no scalability in BeLaRS because it is not possible to use it for specifying large-scale requirements. C-H3₁: There is scalability in BeLaRS because it is possible to use it for specifying large-scale requirements.

Abbreviations: BeLaRS, behavior language requirements specification; VR, virtual reality.

experience. All the students had an average level of experience with requirements specification. There were six females and 25 males aged between 24 and 30.

In this study, two metrics, conformity and usability, were used to evaluate accuracy and usability of the BeLaRS language:

- *Conformity* involves evaluating the accuracy of the keywords used in BeLaRS to represent concepts from the VR domain. In our study, this means analyzing the simplicity, orthogonality, and scalability of BeLaRS. These are three primary metrics used for developing a DSL. In our case, simplicity represents a desirable language that expresses the VR concepts and supports the users in undertaking their work. Orthogonality ensures that each keyword used in BeLaRS represents exactly one distinct concept in the VR domain. Finally, scalability shows how BeLaRS can help specify large-scale requirements for several applications using SG;
- *Usability* seeks to evaluate the use of the BeLaRS language and relies on the technology acceptance model (TAM) [55]. TAM is based on the assumption that the value of technology is determined by the subjects' willingness to use it, ie, the intention to use a technological device is determined by the user's perception of its usefulness and its ease of use. In this context, the TAM variables are based on the following criteria:
 - ease of use refers to the degree to which a person expects the target system to be effortless;
 - a person's intention to use technology determines whether she/he contemplates using it in the future or not;
 - usefulness is defined as a person's subjective feelings about the likelihood that using a specific system will increase their job performance within the VR domain.

Questionnaires were used to ask the volunteers their opinions about conformity and usability of BeLaRS language. According to the metrics presented above, we defined three research questions (C-RQ₁ to C-RQ₃) and their respective null hypotheses (C-Hx₀) and alternative (C-Hx₁), where x represents the number of the RQ (Table 6).

In the light of the points raised above related to usability, we defined five research questions (U-RQ₄ to U-RQ₈) and their respective hypotheses (null and alternative) (Table 7).

TABLE 7 Usability research questions and hypotheses

Research Questions	Hypothesis
U-RQ ₄ : How near is BeLaRS of natural language?	U-H4 ₀ : The requirements specification using BeLaRS is not near of natural language. U-H4 ₁ : The requirements specification using BeLaRS is near of the specification using natural language
U-RQ ₅ : Is it easy to understand the BeLaRS rules?	U-H5 ₀ : Rules of BeLaRS are not easy to understand to specify requirements of VR applications. U-H5 ₁ : Rules of BeLaRS are easy to understand to specify requirements of VR applications.
U-RQ ₆ : Is it easy to specify requirements using BeLaRS?	U-H6 ₀ : BeLaRS does not facilitate requirements specification. U-H6 ₁ : BeLaRS facilitates requirements specification.
U-RQ ₇ : Is it useful to specify requirements for VR application using BeLaRS?	U-H7 ₀ : BeLaRS is not useful to specify requirements for VR applications. U-H7 ₁ : BeLaRS is useful to specify requirements for VR applications.
U-RQ ₈ : Is it quick to specify requirements for VR application using BeLaRS?	U-H8 ₀ : BeLaRS does not quickly specify requirements for VR applications. U-H8 ₁ : BeLaRS quickly specifies requirements for VR applications.

Abbreviations: BeLaRS, behavior language requirements specification; VR, virtual reality.

5.1.2 | Evaluation procedure

This case study consisted of two phases: (1) requirements specification and (2) analysis of the conformity and usability of the BeLaRS from the first phase.

Phase 1—requirements specification

This phase aimed to specify the requirements of a VR application. The output from this phase was a set of requirements specified using BeLaRS. This phase was preceded by two sessions. In the first session, a description with explanations and examples of the rules of BeLaRS was available for volunteers in an online repository.

In the second session, we provided a general description of the system of a VR application that simulates a biopsy examination generated by the virtual medical training (ViMeT) framework [15]. Finally, the volunteers had access to the document in which they performed the requirements specification remotely.

In both sessions, the volunteers had no contact with the person responsible for this experiment. Furthermore, it is worth highlighting that the total time taken by each volunteer to specify the requirements was computed.

Phase 2—analysis of the conformity and usability of the BeLaRS

This phase involved an analysis of the conformity and usability of BeLaRS, which involved conducting a questionnaire to collect the appropriate data. In one question, there was a free-text form, the purpose of which was to get individual feedback from the volunteers. The volunteers had to evaluate each question using the Likert scale [56], which consisted of five criteria. If the majority of students marked 4 or higher on the Likert scale, the answers could be considered to be confirmed. Otherwise, the answers could be regarded as being rejected.

In addition to the results from the questionnaire, we carried out a manual inspection of the requirements to determine which ones (produced in phase 1) conform to BeLaRS and which do not.

5.2 | Experiment to evaluate VR-ViReST approach effectiveness with the ViReST tool

In the second experiment, we conducted a controlled study to assess the effectiveness and applicability of the VR-ReST approach to perform functional testing in VR applications.

We used the GoalQuestionMetric (GQM) model [57], which is divided into five parts to set out the objectives of the experiment:

- Object of study: The objects of study is the VR-ReST approach.
- Purpose: The purpose of this experiment is to evaluate our approach concerning the functional testing in VR programs.

TABLE 8 Definitions of mutation testing to VR-ReST approach

Definitions	Mutation Testing
Original program (<i>P</i>)	Program under test.
Mutant (<i>M</i>)	Faulty versions of the VR program.
Dead mutant (<i>DM</i>)	Mutant that produces different output from the original program for a given test.
Live mutant (<i>LM</i>)	Mutant not killed by the test data.
Equivalent mutant (<i>EQ</i>)	Mutant cannot be killed by any test data because it produces the same visual behaviors as the original VR program.
Mutation operators	Define syntactic changes that generate mutants.
Mutation score (<i>MS</i>)	$MS = \frac{DM(P)}{M(P) - EM(P)}$

Abbreviations: VR, virtual reality; VR-ReST, virtual reality—requirements specification and testing.

- Perspective: This experiment is carried out from the standpoint of the researcher.
- Quality focus: The main effect under investigation is the effectiveness of the VR-ReST approach, which is measured by the number of mutants killed and coverage of test requirements.
- Context: This experiment is performed using three VR programs.

For conducting this experiment, we use the ViReST tool and mutation testing criteria proposed by Hamlet [58] and DeMillo et al [1] to assess the VR-ReST approach. The definitions of mutation testing to the context of this experiment are shown in Table 8.

5.2.1 | Research questions (RQs) and goal definition

According to the information presented in Table 8, we generated test data by ViReST tool to reveal these faults, ie, to distinguish the outputs of the mutated applications from the original one. Finally, the quality of the test set was measured by the mutation score. The following research questions were addressed:

RQ₁: How effective is the VR-ReST approach using the ViReST tool to identify faults in VR programs?

In answer to *RQ₁*, this study collected suitable test data sets for each criterion (NC, EC, EPC, and PPC) implemented in the ViReST tool. Then the effectiveness of the VR-ReST approach was evaluated by the number of failures detected, that in the mutation test, is represented by the number of killed mutants.

RQ₂: Does the quality of system requirements specified correlate with the quality of test data generated per VR program?

The functional testing depends on the quality of the requirement specification. The quality is directly related to the completeness and consistency of the requirements. In our case, the quality of the specified requirements may interfere with the quality of the test data since the test data are generated from these requirements.

To answer *RQ₂*, we analyze if missing specific characteristics and/or requirements is related to the number of test data generated and consequently to the number of mutants that remain alive. We raised the following hypothesis for this research question:

H2₀: There is no relationship between the quality of the requirements specified ($\mu Requirements$) and the quality of the test data generated ($\mu TestData$), thus:

$$H2_0 : \mu Requirements \neg \Rightarrow \mu TestData$$

H2₁: There is a relationship between the quality of the requirements specified ($\mu Requirements$) and the quality of the test data generated ($\mu TestData$); thus,

$$H2_1 : \mu Requirements \Rightarrow \mu TestData.$$

RQ₃: How effective is the VR-ViReST approach for generating test data for detecting faults in VR programs, as compared with random testing?

We compared the VR-ReST approach, using the ViReST tool, with a random testing for fault detection in VR programs. The number of test data generated and the mutation score for both cases were compared with each other. A set of data, according to each criterion, was generated by the ViReST tool to obtain the mutation score. On the other hand, the experiment was performed 10 times using the random testing, and in the end, the mean number of test data generated and the mean of the mutation score were computed. The following hypothesis was defined for this research question:

H3₀: There is no difference in test data generation between the VR-ReST approach ($\mu VR - ReST$) and random testing ($\mu Random$); thus,

$$H3_0 : \mu VR - ReST = \mu Random.$$

TABLE 9 Virtual reality (VR) programs used in the experiments

Programs	LOC	Classes	Functions
ViMeT Game	11.420	41	455
Bouncing Ball	211	1	7
Alternate Appearance	660	3	12

$H3_1$: There is difference in test data generation between the VR-ReST approach ($\mu VR - ReST$) and random testing ($\mu Random$); thus,

$$H3_1 : \mu VR - ReST \neq \mu Random.$$

5.2.2 | Experiment design

In this study, three different programs written in Java language $P = (p_1; p_2; p_3)$ were used (Bouncing Ball, Alternate Appearance, ViMeT Game). The first and the second programs are commonly used in experiments [59], and the last one is a game based on the ViMeT framework [60].

Bouncing Ball is a program that aims to move an object (ball) down, up, right, or left in the scene. The movement of the object occurs by using mouse or keyboard. *Alternate Appearance* is a program that aims to change the appearance (color) of nine objects (balls) in the scene. This appearance can be changed to four colors (white, red, green, and blue) depending on the size of the balls: tiny, small, and large. *ViMeT Game* is an application for medical training based on concepts of serious games and VR simulating for performing breast biopsies [60].

These programs are real and nontrivial VR applications that have been used by a large number of users. Table 9 shows the size of each program. The LOC column represents the number of lines of the source code for each subject. The second and third columns, respectively, represent the number of classes and functions.

This empirical study manipulated an independent variable: test data generation. Test data sets are constructed by a depth-first algorithm to meet each of the test criteria (NC, EC, EPC, and PPC), as explained in Section 2.2. Three dependent variables calculated by the generated test data set were measured:

- Defects detection effectiveness: The effectiveness of the fault detection of a test suite is “successful” when the fault is detected by at least one execution of the test data into a test set, or “a failure” when the fault is not detected by any test data execution;
- Size of test data set: represents the number of test data in the test set;
- Coverage test requirements: In a given VR application, the coverage of the TRs is represented by the requirements satisfied by a test data set T .

5.2.3 | Experiment setup

The procedure followed during the execution of the experiment consists of the following steps, for each of the subject applications:

1. Specifying requirements for applications: We performed the requirements specification $R = \{r_1, r_2, \dots, r_m\}$ for each VR program p_i using the ViReST tool. For each application, a checklist with some questions related to the application was developed in order to facilitate the errors identification. For example, for ViMeT application, errors related to the specification were identified through the checklist produced by the application creator. Once these errors were detected, a new specification was performed and validated again. With the validated specification, it was possible to generate the guideline, which was used as the basis for specifying the requirements in the ViReST tool using BeLaRS. An example of this specification can be seen in Figure 12.
2. Generating test requirements: From the requirements specified, the test requirements $TR = \{tr_1, tr_2, \dots, tr_n\}$ were derived for each test criterion using the ViReST tool.
3. Generating test data set: For the derived test requirements, we used the ViReST tool to generate and to execute test data $T = \{t_1, t_2, \dots, t_r\}$ to check which TRs had been covered. These test data sets generated by the ViReST tool and the covered requirements are stored.
4. Generating mutants for each application: We used MuJava tool* [5] to generate the mutants $M = \{m_1, m_2, \dots, m_q\}$. We only consider mutation operators that modify expression by replacing, deleting, and inserting primitive operators. The MuJava tool provides six kinds of primitive operators: (1) arithmetic operator, (2) relational operator, (3) conditional operator, (4) logical operator, (5) shift operator, and (6) assignment. Furthermore, some of the operators are separated into two or three subdivisions, depending on the number and type of operators. For example, the AOR operator is subdivided into AOR_B (binary), AOR_S (shortcut), and the AOR_U (unary) operator. Table 10 describes these operators.
5. Identifying equivalent mutants: Equivalent mutants were manually identified while designing suitable test sets. These mutants are labeled $EQ = \{eq_1, eq_2, \dots, eq_t\}$. Equivalent mutants are those that have the same visual behavior as the original VR application.
6. Running the test data set: We used the Sikuli tool† [61] for each set of mutants from one subject, to run test data T generated by the ViReST tool until all the mutants were killed. This process was performed by means of a manual transformation of the generated

* MuJava is a mutation system for Java programs. It automatically generates mutants for both traditional mutation testing and class-level mutation testing.

† Sikuli automates anything you see on the screen. It uses image recognition to identify and control GUI components. It is useful when there is no easy access to a GUI's internal or source code.

TABLE 10 Mutation operators used to generate mutants for each application

Operators	Description
AOR_B	Arithmetic operator replacement
AOR_U	
AOR_S	
AOI_U	Arithmetic operator insertion
AOI_S	
AOD_U	Arithmetic operator deletion
AOD_S	
ROR	Relational operator replacement
COR	Conditional operator replacement
COI	Conditional operator insertion
COD	Conditional operator deletion
SOR	Shift operator replacement
LOR	Logical operator replacement
LOI	Logical operator insertion
LOD	Logical operator deletion
ASR	Assignment operator replacement

TABLE 11 Example of a set of test data (TD) generated using the random testing

Requirements	Test Data
1. User selects syringe	TD1: <i>selects, translates, deforms</i>
2. User moves syringe until Breast	TD2: <i>selects, translates, rotates</i>
3. User rotates syringe until Breast	TD3: <i>selects, moves, rotates</i>

test data by the ViReST tool, eg, “1, 2, 2.1, 3, 3.1, 4, 4.1, 5, 5.1, 6, 6.2, 7” into corresponding actions that exist in Sikuli such as “1 = *find*, 2 = *click*, 2.1 = *click*, 3 = *click*, 3.1 = *click*, 4 = *click*, 4.1 = *click*, 5 = *click*, 5.1 = *click*, 6 = *exists*, 6.1 = *exists* and 7 = *findAll*.” After this, we manually added these actions in Sikuli and executed them in each generated mutant. The generated test data aims to verify if the behavior of a mutant program corresponds to the behavior of the original VR program. If the behavior differs from the expected behavior, then the test data are able to identify the mutant.

7. Computing mutation score: The mutation score (MS) of a test set directly indicates its effectiveness on fault detection. We calculated MS for each program p_i as shown in Table 8.
8. Comparison between the VR-ReST approach and random testing: We compared the VR-ReST approach with random testing for generating the test data to assess the suitability of the test data generated. This step is divided into three substeps:
 - (a) We create a database D composed of all possible inputs $IN = \{i_1, i_2, i_3, \dots, i_h\}$, which represent the actions of a given application, eg, “*translates*,” “*deforms*,” and “*moves*,” among others. Each randomly generated test data contains different inputs that are represented by actions as can be seen in the requirements column of Table 11;
 - (b) We developed a random function to generate a data set T from D . For each program p_i , a set of requirements was specified. For each requirement r_i , at least one different input in_i is associated, ie, $in_i \subseteq r_i$. For example, the requirement “*User rotates syringe until Breast*” is associated with the *rotates* input recorded in D . Thus, each test data set T was generated by randomly selecting different inputs from IN that were part of the each requirement $T = in_i \subseteq r_i, \dots, in_h \subseteq r_m$. Table 11 presents an example of three requirements and their respective test data generated using the random testing.
 - (c) We manually added the testing data sets generated in the step (a) to Sikuli and executed them in each generated mutant.

6 | RESULTS AND DISCUSSION

The results of the two experiments are shown following in separate subsections. A descriptive statistical analyses were performed for both experiments.

6.1 | Case study to evaluate BeLaRS effectiveness

To assess the conformity of our approach, we analyzed the simplicity, orthogonality, and scalability of the BeLaRS language (Figure 13), as described in Section 5.1.

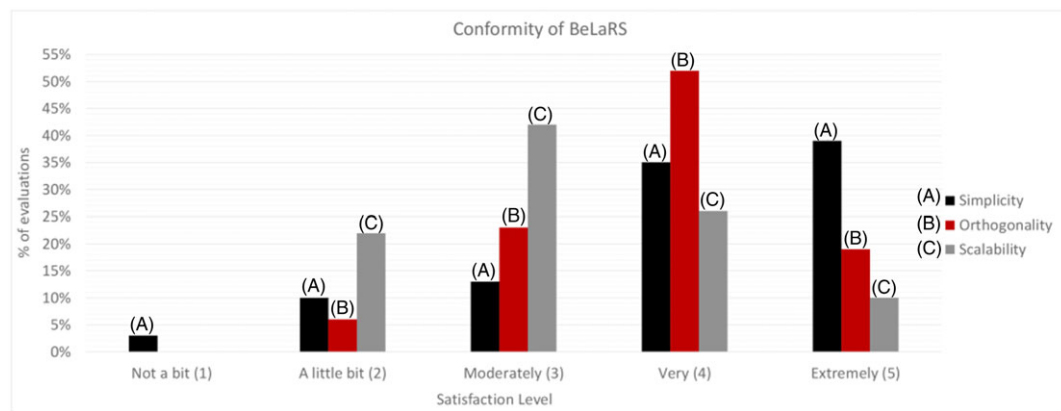


FIGURE 13 Conformity of behavior language requirements specification (BeLaRS) based on simplicity, orthogonality, and scalability metrics according to Likert scale

6.1.1 | Simplicity in BeLaRS (C-RQ₁)

Seventy-four percent of the subjects assessed the BeLaRS language as very or extremely simple, which confirmed the alternative hypothesis C-H₁. Otherwise, 13% of the volunteers evaluated the simplicity of the BeLaRS language as “moderately,” 10% as “a little bit,” and 3% as “not a bit”; these volunteers—26% of the total—thought that keywords are not sufficient to assist users in the requirements specification since these keywords are only related to SG concepts. We believe that some volunteers do not have enough knowledge about the SG structure, which justifies these assessments.

6.1.2 | Orthogonality in BeLaRS (C-RQ₂)

Seventy-one percent of the volunteers evaluated the BeLaRS language as very or extremely orthogonal, which confirms the alternative hypothesis C-H₂. The volunteers provided satisfactory feedback for this question, which showed that the keywords of BeLaRS represent distinct concepts in VR. However, it was clear from the feedback of some volunteers that some keywords can have different connotations.

6.1.3 | Scalability in BeLaRS (C-RQ₃)

Forty-two percent of the volunteers evaluated BeLaRS as “moderately” scalable, which rejects alternative hypothesis C-H₃. Seven volunteers (22%) assessed BeLaRS as “little bit” scalable and stated that some of the BeLaRS rules need to be improved. Only 36% of the subjects evaluated BeLaRS as scalable. Thus, it is clear that the lack of scalability is because the keywords database was only created to cover some of the VR domains.

As shown above, most of the alternative hypotheses were confirmed. This means that, according to the volunteers, BeLaRS has a precise syntax and comprises a reasonable number of keywords that can support the requirement specification for VR applications. However, we had to reject the alternative hypothesis of research question (C-RQ₃), since the scalability needs to be improved to overcome some of its limitations. We believe that the addition of more keywords will increase the number of VR applications that can use BeLaRS. Given this, it is important to include new keywords, mainly related to features of colors and behaviors, such as open, close, walk, and run, in order to provide a broad range of possibilities during the specification.

6.1.4 | Usability

The usability of BeLaRS, presented in Figure 14, was verified by the responses of the volunteers in relation to time, usefulness, easiness of specification, and comprehensibility of rules.

6.1.5 | BeLaRS closeness to the natural language (U-RQ₄)

Seventy-four percent of the volunteers evaluated BeLaRS as close to a natural language, which confirms alternative hypothesis U-H₄. Twenty-three percent of the volunteers assessed BeLaRS as “moderately” to a natural language, and only 3% chose “little bit.”

6.1.6 | Understanding of the BeLaRS' rules (U-RQ₅)

Seventy-one percent of the volunteers assessed the BeLaRS rules (U-H₅) as “extremely” and “very” easy to understand. Only 3% evaluated the rules as “little bit” and “not a bit” easy to understand. We noticed that this 3% was due to a lack of clarity in the explanation and logical meaning of the rules of BeLaRS.

The volunteers' little experience in VR explained that their evaluations were attributed to the BeLaRS rules since these rules were not easy to understand and were not useful compared with those of the natural language.

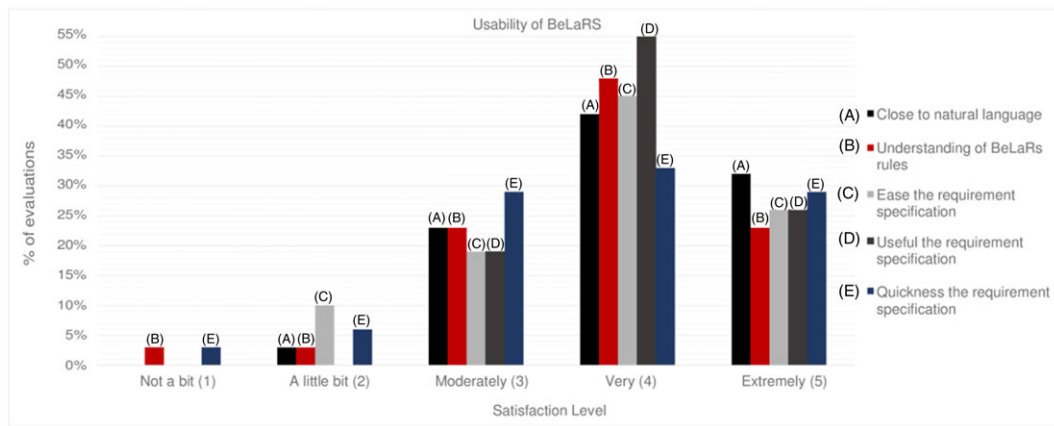


FIGURE 14 Usability of behavior language requirements specification (BeLaRS) based on volunteers' opinion about aspects related to proximity to natural language, understanding rules, ease, utility, and quickness in the requirement specification

TABLE 12 Descriptive statistical of the spent time with and without outliers

Statistical Measures	With Outliers	Without Outliers
Min	11	14
Max	45	29
Average	22.52	21.72
Median	23	23
Standard deviation	7.99	1.62

6.1.7 | Requirements specification using BeLaRS (U-RQ₆)

Seventy-one percent of the volunteers confirmed that it is easy to specify requirements using BeLaRS (Figure 14). However, three volunteers (10%) stated that it is only “a little bit” easy to specify requirements using BeLaRS. This assessment may be due to some uncertainty caused by the control structures and restrictions—which are included in the rules—as well as how they were explained.

6.1.8 | Useful to specify requirements using BeLaRS (U-RQ₇)

Eighty-one percent of the volunteers confirmed that BeLaRS was a useful means of specifying requirements for VR applications. Thus, the alternative hypothesis $U - H7_1$ is confirmed. It is worth stressing the importance of BeLaRS and the assistance it provides in the requirements specification with a formalized specification and hence reduces the risk of misinterpretation, ambiguity, and possible errors.

6.1.9 | Quickness in the requirements specification using BeLaRS (U-RQ₈)

The volunteers performed the requirements specification in a time interval ranging from 11 to 45 minutes. As can be seen in Table 12, the average time was 22.52 min.

We observed that six outliers had a time far above or below average. These time differences may have been caused by variations in the skills of each student in the VR area and their ability to understand the rules of BeLaRS. The three fastest volunteers are experts with over 4 years of experience in the development of VR applications, while the slower ones had no experience in the VR area.

Table 12 presents a descriptive statistic of the spent time to requirements specification before and after the exclusion of the outliers. It is possible to notice the outliers influence in the average time spent on the requirements specification. With the outliers exclusion, the standard deviation is reduced of 7.99 to 1.62. On the other hand, the median was the same for both groups (with outliers and without outliers), evidencing that the times of the outliers not tend the results for the upper and lower margins.

As a result of this experiment, some issues were revealed, contributing for improvement of BeLaRS such as

- application of the sentence structure grammar theory proposed by Chomsky [45];
- grammar reduction, making it simpler and more direct to specify requirements;
- definition of multiple objects of the same type using an “ID” represented by a digit. An example is shown in Figure 15;
- inclusion of behaviors on VED description: *moves up*, *moves down*, *moves right*, *moves left*, *moves out*, *moves into*, *moves diagonal*, *picks*, *drops* (Figure 15);
- inclusion of prepositions: *up to*, *in*, *on*, *between*, as can be seen in the fourth sentence of *Main path* in Figure 15.

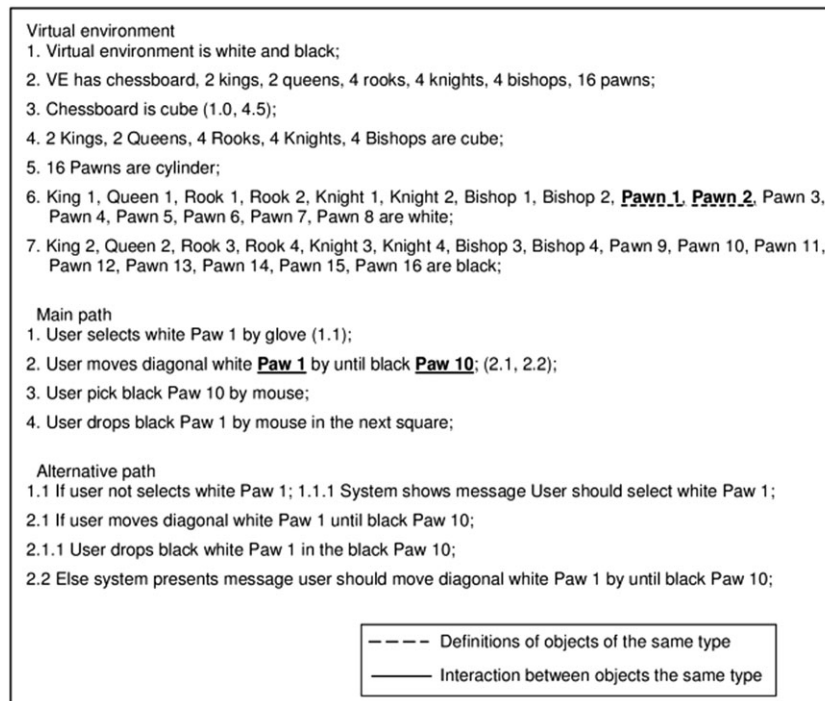


FIGURE 15 An example of requirements specification adopting improved behavior language requirements specification (BeLaRS). The improvements consisted in (1) inclusion of new behaviors and prepositions to facilitate the description of virtual environment (VE) (lines 2, 3 and 4 of Main path); (2) definition of the multiple objects of the same type, which are differentiated through the inclusion of digits highlighted in bold (lines 6 and 7 of VE); and (3) interaction between objects of the same type (line 2 of Main path)

TABLE 13 Number of mutants generated and equivalents identified for each program

Programs	Mutants	Equivalents
Bouncing Ball	53	13
Alternate Appearance	209	65
ViMeT Game	1077	613

At the light of these results, the language was improved in a new version, simpler, and more complete than the one outlined in Section 3.2. This new version was used in the second experiment.

6.2 | Experiment to evaluate the VR-ViReST effectiveness by using the ViReST tool

This section examines the results achieved regarding effectiveness from our approach to fault detection, based on the procedures outlined in the Subsection 5.2.

6.2.1 | Effectiveness of the VR-ReST approach to identify faults (RQ₁)

The numbers, regarding the mutants generated for each system, are shown in Table 13. The *Programs* column shows the name of the programs. *Mutants* column shows the total number of mutants generated from the traditional operators by MuJava, and the *Equivalents* column displays the number of equivalent mutants, ie, those mutants that always have the same visual behavior as the original application. The equivalent mutants were manually identified.

Table 14 shows the test data generated according to each test criterion and the score obtained from the generated mutants presented in Table 13. The *Programs* column presents the programs names. The *Req.* column lists the number of requirements specified (functionality) of each program. The *TD* column presents the number of test data generated by the VR-ReST approach, according to each criterion (NC, EC, EPC, and PPC). Finally, the *MS* column provides the mutation score per each test criterion. The results indicate that the VR-ReST approach is effective to detect faults since, for the weakest criterion, from 87.79% to 92.38% of the faults were revealed and, for the strongest criterion, the range goes from 89.67% to 93.83% of the faults revealed.

In a similar domain, Oliveira et al [62] performed an empirical analysis of the effectiveness of a set of 18 GUI (graphical user interface) based on mutation operators (MOs). Such study reveals that the GUI MOs had almost 84% of effectiveness. Therefore, for the VR domain, the MS achieved by VR-ReST approach can be considered a good result.

TABLE 14 Number of data test generate and mutation score (MS) per criterion for each program

Programs	Req.	TD				MS, %			
		NC	EC	EPC	PPC	NC	EC	EPC	PPC
Bouncing Ball	11	2	2	3	3	87.79	87.19	88.64	89.67
Alternate Appearance	33	2	2	4	6	95.27	94.56	95.83	96.41
ViMeT Game	37	2	3	3	3	92.38	92.89	93.45	93.83

Abbreviations: EC, edge coverage; EPC, edge-pair coverage; NC, node coverage; PPC, prime path coverage; TD, test data.

1. User clicks on the Go button 2. System moves the ball slowly up and down 2.1 If the system does not move the ball 2.1.1 System stop 2.2 Else system keeps moving ball 3. User selects key S 3.1 If the user selects key S 3.1.1 Ball moves to the right 3.2 Else system present the message: Ball moved to the wrong side 4. User key selects key A 4.1 If the user selects key A 4.1.1 Ball moves to the left 4.2 Else system present the message: Ball moved to the wrong side	(A)
1. User clicks on the Go button 2. System moves the ball slowly up and down 2.1 If the system does not move the ball 2.1.1 System stop 2.2 Else system keeps moving ball <u>slowly up and down</u> 3. User selects key S 3.1 If the user selects key S 3.1.1 Ball moves to the right <u>inside of the scene</u> 3.2 Else system present the message: Ball moved to the wrong side 4. User key selects key A 4.1 If the user selects key A 4.1.1 Ball moves to the left <u>inside of the scene</u> 4.2 Else system present the message: Ball moved to the wrong side	(B)

FIGURE 16 A, Original specification and B, modified specification of the Ball application used to evidence relationship between the quality of requirements specified and the quality of test data generated

We observe that the VR-ReST approach enables to identify failures during the development process. For the Bouncing Ball application, for example, the ball must move up and down inside the scene. In this application, the fault identified was that the ball moves erroneously leaving the delimited scene. To identify the faults in VR applications, commonly, the usability testing is performed at the end of the development process. However, this testing may not reveal particular types of faults for being a user-centered technique. Therefore, the VR-ReST approach allows detecting faults during the development process verifying whether the application conforms to the specified requirements.

6.2.2 | Relationship between the quality of requirements specified and the quality of test data generated (RQ₂)

The quality of the specified requirements and the test data generated are directly related. A good requirement is one that completely and unambiguously describes the functionality of the program with all its characteristics. A set of good test data is one capable of revealing existing faults, ie, killing all nonequivalent mutants.

In this context, it is important to note that the living mutant is directly related to the requirements specification since the *TRs* are derived from these requirements and the test data *T* are generated to cover the *TRs*. Thus, regardless of the criteria used, if a particular requirement r_i is not part of the specification, then it will be impossible to produce a test data t_i to cover r_i generated from requirements specification.

In order to corroborate the results presented in Table 14, a new specification was performed from the requirements of the Bouncing Ball, Alternate Appearance, and ViMeT Game programs to kill the identified living mutants. Figure 16A,B presents two specifications: the original specification and the modified specification of the Bouncing Ball application, respectively.

TABLE 15 Relationship between the quality of the requirements specified and the quality of the test data generated from the modified specifications

Programs	Req.	Req. Modified	TD				MS, %			
			NC	EC	EPC	PPC	NC	EC	EPC	PPC
Bouncing Ball	11	3	2	2	3	3	100	100	100	100
Alternate Appearance	33	4	2	2	4	6	97.34	97.34	99.68	100
ViMeT Game	37	3	2	3	4	4	97.47	97.92	98.56	98.89

Abbreviations: EC, edge coverage; EPC, edge-pair coverage; NC, node coverage; PPC, prime path coverage; TD, test data.

TABLE 16 Descriptive statistic summarizing the characteristics of the specifications of the Bouncing Ball program

Statistics	Bouncing Ball							
	NC		EC		EPC		PPC	
	(O)	(M)	(O)	(M)	(O)	(M)	(O)	(M)
Min	87.19	100	87.02	100	88.12	100	89.01	100
Max	88.03	100	87.50	100	89.43	100	91.07	100
Mean	87.79	100	87.19	100	88.64	100	89.67	100
Median	87.91	100	87.12	100	88.51	100	89.31	100
Std Dev	0.42	0	0.21	0	0.63	0	0.96	0

Abbreviations: EC, edge coverage; EPC, edge-pair coverage; NC, node coverage; PPC, prime path coverage.

TABLE 17 Descriptive statistic summarizing the characteristics of the specifications of the Alternate Appearance program

Statistics	Alternate Appearance							
	NC		EC		EPC		PPC	
	(O)	(M)	(O)	(M)	(O)	(M)	(O)	(M)
Min	95	97	94	97	95.34	99.12	95.81	100
Max	96	98	95.15	98	96.22	99.95	96.91	100
Mean	95.27	97.34	94.56	97.34	95.83	99.68	96.41	100
Median	95.05	97.19	94.56	97.18	95.90	99.82	96.46	100
Std Dev	0.48	0.44	0.59	0.45	0.37	0.38	0.46	0

Abbreviations: EC, edge coverage; EPC, edge-pair coverage; NC, node coverage; PPC, prime path coverage.

As shown in Figure 16B, the modified requirements are 3.1.1 and 4.1.1. In the modified version, some features, which are underlined, have been added. These characteristics represent behaviors of the ball controlled by the system. The missing characteristics in Figure 16A are directly related to the living mutants.

A new specification of requirements for each program was carried out by VR-ReST approach in an attempt to kill the live mutants. After the specification, new test data were obtained according to each criterion. The results from the new specification are presented in Table 15.

Analyzing Tables 14 and 15, it is possible to verify that for the programs Bouncing Ball and Alternate Appearance only the test data presented in Table 14 were needed to cover the new requirements and, hence, to kill the living mutants. On the other hand, for the ViMeT Game program, two more test data (one test data for the EPC and one for the PPC) were needed to kill 25 of the 32 live mutants.

Tables 16, 17, and 18 present summarized statistics over the Bouncing Ball, Alternate Appearance, and ViMeT Game program, respectively. We found that in the Bouncing Ball program, on average (mean), 88.32% of MS was achieved with original (O) specification and 99.75% with modified (M) specification (Table 16). For Alternate Appearance, 95.51% was achieved with the original specification and 98.59% with the modified specification (Table 17). And for ViMeT, 93.13% was achieved with the original specification and 98.21% with the modified specification (Table 18). Both the mean and median of the mutation scores of programs using modified specifications suggest that the quality of the specified requirements influences in the quality of the test data set generated.

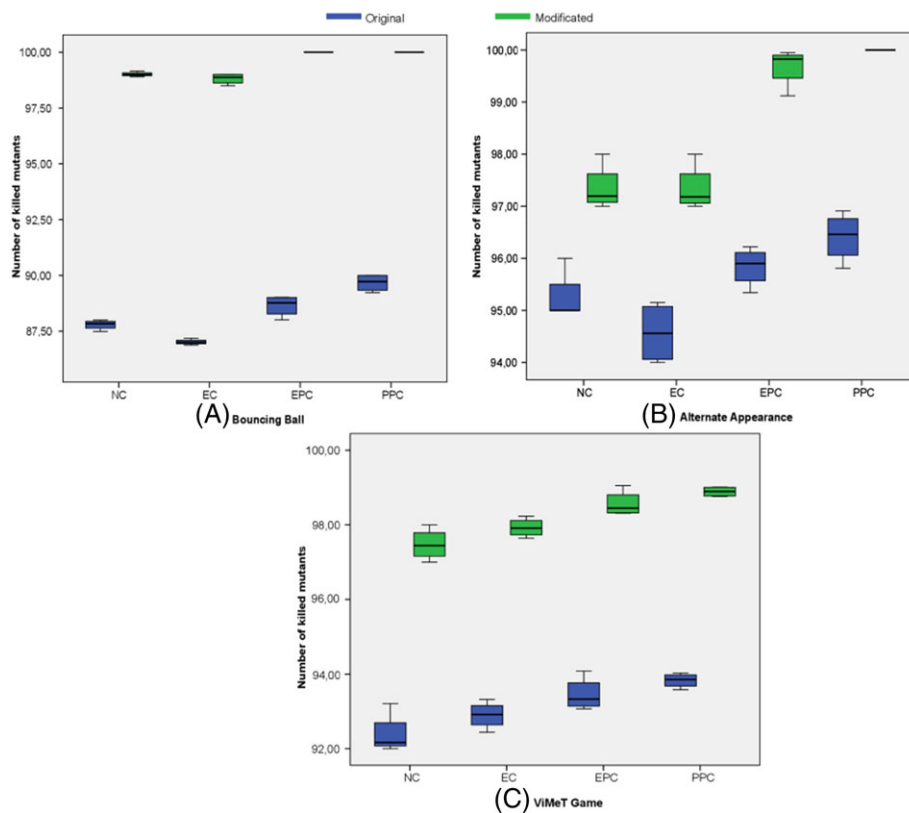
Figure 17 summarizes the data from Tables 16, 17, and 18 in three boxplots, respectively. The thick lines are the medians, ie, the mutation score of each specification (original and modified) using the four testing criteria per program.

In Figure 17, the median of MS using the original version is 88.21% while using the modified version is 100%. The boxes around the medians enclose the 25th (Q1) through the 75th (Q3) percentiles, and the “whiskers,” the lines above and below the boxes, represent the “range” of the data. These boxplots very clearly suggest that all programs using the (M) specification have higher MS than the (O) specification.

TABLE 18 Descriptive statistic summarizing the characteristics of the specifications of the ViMeT Game program

Statistics	ViMeT Game							
	NC		EC		EPC		PPC	
	(O)	(M)	(O)	(M)	(O)	(M)	(O)	(M)
Min	92	97	92.44	97.64	93.07	98.31	93.58	98.76
Max	93.21	98	93.32	98.23	94.08	99.05	94.03	99.01
Mean	92.38	97.47	92.89	97.92	93.45	98.56	93.83	98.89
Median	92.16	97.44	92.91	97.91	93.33	98.44	93.85	98.89
Std Dev	0.55	0.42	0.36	0.25	0.44	0.34	0.19	0.13

Abbreviations: EC, edge coverage; EPC, edge-pair coverage; NC, node coverage; PPC, prime path coverage.

**FIGURE 17** Boxplots giving an overview of the distribution of the mutation score (MS) obtained using each criteria for original specification (O) and modified specification (M). A, Boxplot summarizing the MS obtained for the Bouncing Ball program. B, Boxplot summarizing the MS achieved for the Alternate Appearance program. C, Boxplot summarizing the MS obtained for the ViMeT Game program

In this question, we notice that the quality of the test data generated is dependent on a complete requirements specification. To confirm this dependence, we performed this experiment in twofold parts.

In the first part, we used the (O) requirements specification to generate the test data and to compute the MS per program. In the second part, we analyzed the remaining mutants to associate their faults with the specified requirements. From this analysis, we observe that for some faults, there is no a particular requirement in the (O) specification.

To confirm that a complete requirements specification (ie, addressing all possible interactions and characteristics of VE and objects) can generate a high-quality test data set, we performed the following steps: (1) We included the missing characteristics in the requirements associated with the remaining mutants, (2) we generated a test data set, and (3) we computed the MS.

Therefore, as can be seen in Figure 16, from the inclusion of the missing characteristics in the requirements, it was possible to increase the MS by 11.43% for the Ball program, 3.08% for Alternate Appearance, and 5.08% for ViMeT Game.

6.2.3 | Effectiveness of VR-ReST compared with random testing to identify faults (RQ₃)

In this research question, a comparison was made between the VR-ReST approach and the random testing to evaluate the quality of test data generated and the MS obtained for each program. To carry out this comparison, we performed two analysis: (1) using an equal size test data set and (2) using test data sets with different sizes.

First, we investigated if the random testing is able to obtain a similar coverage to that of the VR-ReST approach using the same size of test data set generated by the VR-ReST approach. This was carried out for each program adopting the test data generated by the VR-ReST approach presented in Table 15.

Table 19 provides data about the effectiveness of both approaches using a same size test data set. This table shows, for each program, the number of test data set generated by each test criteria (NC, EC, EPC, and PPC) presented in Table 15 (*Test Data* column), the mutation score for our approach (*MS ViReST* column), and the mutation score for random testing (*MS Random* column) using the same size of test sets presented in the *Test Data* column. In all programs, it was possible to observe that with the same size of the test data set, the VR-ReST approach obtained better mutation scores than the random testing.

The size of the test data set to be used by the random testing was defined by analyzing the test data median generated by the VR-ReST approach criteria. The median value of each application was defined as the standard size for the generation of test data using the random testing. We performed this comparison to verify if the random testing would be able to achieve the same or similar mutation score using the same amount of test data generated by the VR-ReST approach. Therefore, these results suggest that the random testing achieves a lower MS once it does not use test criteria to generate the test data set.

Finally, we compared the VR-ReST approach with random testing by evaluating the number of the test data generated capable of killing the largest number of mutants.

Table 20 summarizes the number of test data generated using each approach for the three VR programs. The program with the most test data for all the criteria was Alternate Appearance: two test data for NC, two for EC, four for EPC, and six for PPC. The program with the least test data was Bouncing Ball, which had two test data for NC, two for EC, three for EPC, and three for PPC. As expected, the more elaborated criteria required more test data than the simpler ones, since the former subsumes the later. PPC required around 21.43% more test data than EPC, 50% more test data than EC, and 57.15% more than NC.

As verified in Table 20, test data generation using the VR-ReST approach is more efficient than random testing, since it generates less test data with high quality able to kill 98.91% of the mutants. The third and fifth columns of Table 20 summarizes the MS reached using the test data set generated by the VR-ReST approach and random. Even with more test data, the MS obtained with random generation is lower than using VR-ReST.

For this experiment, the test cases were generated until, for three consecutive generations, it was not possible to improve the mutation score. For instance, the set for Bouncing Ball is 8. It means that, in our generation, adding the ninth, 10th, and 11th test case would not improve mutation score. Therefore, we performed this comparison to verify whether the random testing would be able to achieve the same or similar mutation score using distinct amount of test data generated by the VR-ReST approach.

According to the test data generated and to MS obtained presented in Table 20, we observed that the random testing obtained an increase in the mutation score, compared with Table 19. However, the random testing did not reach 100% mutation score and neither did they achieve a score similar to the other criteria even though it produces more test data to cover the TRs.

In most cases, the generation of test data using random testing is time-consuming with no guarantee of producing a test data set that can meet some of the key requirements, especially for large programs. Therefore, by analyzing all the applications, the results indicated that the test data generated by the VR-ReST approach are more adequate than the data generated by the random testing, since the random testing achieved mutation scores of 83% to 89%, while the VR-ReST approach achieved mutation scores of 98% to 100%.

TABLE 19 Comparison between the MS achieved by the VR-ReST approach and random testing using a same size test set

Programs	Test Data				MS VR-ReST, %				MS Random, %			
	NC	EC	EPC	PPC	NC	EC	EPC	PPC	NC	EC	EPC	PPC
Bouncing Ball	2	2	3	3	100	100	100	100	77.85	77.95	79.43	79.95
Alternate Appearance	2	2	4	6	97.34	97.34	99.68	100	77	77.04	78.02	79
ViMeT Game	2	3	4	4	97.47	97.92	98.56	98.89	81.35	81.76	83.25	84.47

Abbreviations: EC, edge coverage; EPC, edge-pair coverage; MS, mutation score; NC, node coverage; PPC, prime path coverage; VR-ReST, virtual reality—requirements specification and testing.

TABLE 20 Comparison between the MS achieved by the VR-ReST approach and random testing using a different size test set

Programs	Test Data VR-ReST				MS VR-ReST, %				Random Set Size	MS Random, %
	NC	EC	EPC	PPC	NC	EC	EPC	PPC		
Bouncing Ball	2	2	3	3	100	100	100	100	8	83.49
Alternate Appearance	2	2	4	6	97.34	97.34	99.68	100	14	84.21
ViMeT Game	2	3	4	4	97.47	97.92	98.56	98.89	11	89.22

Abbreviations: EC, edge coverage; EPC, edge-pair coverage; MS, mutation score; NC, node coverage; PPC, prime path coverage; VR-ReST, virtual reality—requirements specification and testing.

From the analysis of the results, we conclude that the VR-ReST approach can be considered a feasible alternative to assist the test in the VR applications development process. The benefits from using the VR-ReST approach includes four test criteria that can achieve different test requirements; this means that the test data set generated by our approach can help to detect several kinds of faults.

6.3 | Threats to validity

This paper is concerned with effective automatic test data generation using test criteria for VR applications.

One possible threat to the validity of the results comes from the generalization of the programs employed. This is a common threat in software engineering studies, and the risk was mitigated by selecting a set of applications that could represent programs of different sizes and domains. In the first experiment, the size of the application, ie, the number of requirements, was limited. In this study, the requirements were specified by graduate volunteers who had a basic knowledge of software engineering. However, only three volunteers had industrial experience in the VR area, which may have limited the results as regards the conformity and usability of the BeLaRS language.

In the second experiment, we used three different applications. These applications were chosen from existing works in this area. We believe that our results can be generalized to the class of applications that uses scene graph concepts since these applications are of different domains of VR.

Additionally, in this second experiment, we analyzed the approach adopted for the test data generation. This approach was implemented using a depth-first search algorithm based on structural testing criteria. While we have extensively tested our experimental approach, it is possible that there are faults that could lead to incorrect conclusions. However, to mitigate the risk of bias and to increase the confidence level, we analyzed the data in tables and graphs as well as by statistical tests to ensure that the results are significant and correctly interpreted and not simply generated by chance.

Threats to internal validity define the level of confidence in the results, expected and obtained. Concerning the first experiment, the volunteers did not know we had developed BeLaRS. For this reason, we urged the students to perform the requirements specification using BeLaRS and then assess it objectively.

Threats to validity by construction were overcome by adopting several threat mitigation strategies in the case study, such as the use of documents with explanations, and an example of BeLaRS' applicability was provided to help the volunteers become familiar with the language.

7 | RELATED WORK

Although for several years, a great deal of attention has been paid to the study of software testing; only a few researchers have addressed the issue of automated testing for the VR domain. The main studies of software testing for VR applications that have appeared in the literature aim to propose approaches for supporting testing techniques, testing interfaces, and usability testing.

[28] and [63] address two approaches to assist the structural testing and functional testing respectively. Bezerra et al [28] proposed four testing criteria (All-Nodes-Leaves, All-Nodes-Intermediary, All-Paths-Ascendants, and All-Paths-Descendants) for VR applications based on structural testing criteria and the scene graph. These criteria are used to derive the test requirements that are based on the SG of each application using a tool named VETesting. This tool assists the construction and correction of the SG, but the testing performed by the VETesting tool is not automated.

Florczyk and Winiecki [63] describe a parametric method for functional testing that allows tests to be carried out on the virtual application during the development phase of the user interface. This method assumes that the user may firstly act unconventionally during the operation of the virtual application and may not always apply the instructions included in the manual of the device (the measurement scenario).

The studies by Bierbaum et al [29] and Chen et al [30] present different proposals to test interfaces and usability, respectively. Bierbaum et al [29] proposed a framework for supporting the testing of interfaces that allows developers to employ standard software engineering techniques that require unit testing methods. Finally, Chen et al [30] proposed an approach called modified group usability testing (MGUT), to test the usability of a nonimmerse VR-based system.

In relation to the requirements specification process, there are twofold lines of research: proposals with manual approaches and automated proposals. The manual proposals involve defining models/methods to facilitate the mapping of requirements in design specifications to result in an implementation compatible with the previously identified requirements. In this context, Salem [64] describes a methodology called human factors (HF), used to specify functional requirements to support risk analysis in VE. Scaife and Rogers [26] employed a user-centered design method that can be used when dealing with specific issues concerned with the properties of virtual environments. Tanriverdi and Jacob [25] introduced the VRID (virtual reality interface design) in which the requirements of the interface are translated into design specifications that can be implemented by software developers. Pellens et al [24] adopted an approach called VR-WISE using conceptual specifications to improve the VE reusability, extensibility, and modularity. Finally, Seo and Kim [23] proposed CLEVR (concurrent and level by the level development of VR system) a model that considers the functions, behavior, hierarchical modeling, user task and interaction modeling, and composition reuse of VR objects.

As for the automated proposals, only the work by Kim [22] fits in this line. Kim [22] describes a specification method incorporating form design in addition to behavior and function using known diagrams of software engineering (data flow diagrams [DFD] and state charts), an object-oriented form representation called the visual object specification (VOS). This method is embedded in a tool called the ASADAL_PROTO. This tool aims to simulate the specifications. For carrying out the simulation, the specifications are interpreted, activating and rendering VOS objects, for verifying or validating their intended behavior and functionality.

Other contributory factors for designing games involve making formal descriptions for user interface design [65], the behavioral aspects of interactive systems [66], and the behavior of multitouch systems [67]. These approaches adopt an interactive cooperative objects (ICO) language recommended by Navarre et al [65], which focuses on the user interface design.

Although many points have been addressed, no methodology has proposed a semiformal language to support the specification of requirements that will be used as input to perform the functional test. Therefore, our approach automates functional testing for virtual reality applications. We created a model for this that employs an incremental technique to guide the RE process (requirement elicitation, requirement specification, and requirement validation) together with a semiformal language that provides support for written requirements based on SG concepts and use cases. From these requirements written using BeLaRS language, our approach makes it possible for the test requirements and test data to be automatically generated using four test criteria.

8 | CONCLUSIONS

An approach to functional testing that generates test data from requirements specified through a semiformal language for VR applications was proposed. It employed a case study to assess the effort involved to enhance the effectiveness of BeLaRS in carrying out the requirements specification of VR applications. Secondly, an experiment with three different applications was performed to assess the effectiveness and applicability of the ViReST for test data generation.

The first experiment evidenced the conformity and usability of BeLaRS concerning the process of the requirements specification of VR applications. The effectiveness of BeLaRS was established by the scores it achieved in conformity and usability. Roughly, 74% of the volunteers confirmed that BeLaRS employs the VR concepts and supports the users and stakeholders in carrying out their work, and 81% of the subjects confirmed that the logical structure of BeLaRS is close to natural language; it is thus a useful and easy means for ensuring the requirement specification of VR applications.

Commonly in the VR domain, RE (design, development, and testing) faces two main challenges, (a) interpreting and (b) handling elicited requirements. However, so far, there has been no approach/tool that formalizes the requirements or can mitigate the problems arising from the issues mentioned above. We therefore believe that BeLaRS can support the requirement specification by a semiformal method. The reason for this is that the simplicity and expressive power of BeLaRS can simplify the interpretation and handling of functional testing.

In the second study, we used mutation testing criteria to assess the effectiveness of the VR-ReST approach by examining the generation of test data in the light of the four criteria (NC, EC, EPC, and PPC). The method employed proved to be effective in its attempt to automate test data generation from the requirements specifications using BeLaRS. This was achieved by transforming each requirement specified into a CFG node. Covering all the nodes in this graph is equivalent to covering all the test requirements. The four structural criteria have effectiveness scores. Roughly for the three VR programs, NC killed on average 98.27% of the mutants, EC 98.42%, EPC 99.41%, and PPC 99.63%. However, these numbers may be deceptive because the existence of a large number of redundant mutants helps to increase the mutation score.

In the case of the number of test data generated or correlated, we observed moderate correlations to the number of requirements since the test data set generated depended on the specified requirements. In the ViMeT Game program, for example, we had to include new requirements in the specification, and two further test data sets were generated from these requirements, able to kill the living mutants.

When our approach is compared with random testing regarding the adequacy of test data set and effectiveness, the results indicate that there is a significant difference between the test data generated through the VR-ReST approach and random testing. We observed that with test sets of the same size, random testing obtained lower mutation score when compared with VR-ReST. Even when was allowed to generate larger test sets, random testing did not reached the same mutation scores of VR-ReST.

Regarding effectiveness, we compared the mutation scores obtained by the VR-ReST approach and random testing using the same test data size generated by the VR-ReST approach. Table 19 summarizes the results of this comparison. Thus, it was clear that random testing was unable to kill the whole set of mutants since most of the test data produced are not sufficient to meet specific requirements and there is no guarantee that suitable test data can be generated by complying with any testing criteria. These results demonstrate that the use of test criteria by VR-ReST approach helps to achieve a higher mutation score than random testing. The results indicate that VR-ReST obtained on average an MS 20% higher than random testing, with test sets of the same size. Regarding the number of test data, VR-ReST tends to create smaller sets when compared with the random testing.

Thus, our results suggest that our approach is promising for the following reasons: It assists in the writing and validation of the requirement, reduces the risks posed by requirements specification by employing a semiformal language, and produces an automatically effective test data set, based on requirements, and able to detect different faults both before and during the development of the VR applications.

In addition, as future work, we intend to improve BeLaRS by including more keywords from different VR domains. In addition an evaluation should be performed in the VR-ReST approach using a search-based technique to generate test data.

ACKNOWLEDGMENTS

The authors thank the Brazilian National Council of Scientific and Technological Development (CNPq), processes 559931/2010-7, 559915/2010-1, and 305999/2016-8; São Paulo Research Foundation (FAPESP), process number 2012/06476-4, 2010/09806-0, and 2010/15691-0; and National Institute of Science and Technology - Medicine Assisted by Scientific Computing (INCT-MACC).

ORCID

Alinne C. Corrêa Souza  <http://orcid.org/0000-0002-2525-7801>

REFERENCES

1. R. A. DeMillo, R. J. Lipton, and F. Sayward, *Hints on test data selection: Help for the practicing programmer*, Computer. **11** (1978), no. 4, 34–41.
2. S. Rapps and E. J. Weyuker, *Selecting software test data using data flow information*, IEEE Trans. Software Eng. **SE-11** (1985), no. 4, 367–375.
3. R. A. DeMillo, D. S. Guindi, W. M. McCracken, A. J. Offutt, and K. N. King, *An extended overview of the Mothra software testing environment*, Proceedings of the 2nd Workshop on Software Testing, Verification, and Analysis, Los Alamitos, 1988, pp. 142–151.
4. M. E. Delamaro, J. C. Maldonado, and A. M. R. Vincenzi, *Proteum/IM 2.0: An Integrated Mutation Testing Environment*. In: W. E. Wong (ed.) *Mutation Testing for the New Century*. The Springer International Series on Advances in Database Systems, vol. 24. Springer, Boston, MA, 2001, pp. 91–101.
5. Y. S. Ma, A. J. Offutt, and Y. R. Kwon, *MuJava: An automated class mutation system*, Software Testing, Verification Reliab. **15** (2005), no. 2, 97–133.
6. S. Boukhris, A. Andrews, A. Alhaddad, and R. Dewri, *A case study of black box fail-safe testing in web applications*, J. Syst. Software. **131** (2017), 146–167.
7. A. Andrews, S. Boukhris, and S. Elakeili, *Fail-safe testing of web applications*, Proceedings of the 23rd Australian Software Engineering Conference (ASWEC), Sydney, New South Wales, Australia, 2014, pp. 200–209.
8. N. Alshahwan and M. Harman, *Automated web application testing using search based software engineering*, Proceedings of the 26th International Conference on Automated Software Engineering (ASE), Lawrence, Kansas, USA, 2011, pp. 3–12.
9. K. Pan, X. Wu, and T. Xie, *Guided test generation for database applications via synthesized database interactions*, ACM Trans. Software Eng. Method. **23** (2014), no. 2, 12.
10. E. Rogstad and L. Briand, *Cost-effective strategies for the regression testing of database applications: Case study and lessons learned*, J. Syst. Software. **113** (2016), 257–274.
11. F. Belli, M. Beyazit, C. Budnik, and T. Tuglular, *Advances in automated model-based system testing of software applications with a GUI Front-End*, Advances in Computers. **80** (2010), 121–162.
12. E. Kowalczyk and A. Memon, *Extending manual GUI testing beyond defects by building mental models of software behavior*, Proceedings of the 30th International Conference on Automated Software Engineering Workshop (ASEW), Lincoln, NE, 2015, pp. 35–41.
13. G. Burdea and P. Coiffet, *Virtual reality technology*, 2nd ed., Wiley-IEEE Press, New Brunswick, 2003.
14. T. G. Kirner and V. Martin, *A model of software development process for virtual environments*, Proceedings of IEEE Symposium on Application-Specific Systems and Software Engineering and Technology (ASSET), Richardson, TX, USA, 1999, pp. 155–161.
15. A. C. M. T. G. Oliveira and F. L. S. Nunes, *Building a open source framework for virtual medical training*, Digit. Imaging. **23** (2010), 706–720.
16. M. Cote, J. Boulay, B. Ozell, H. Labelle, and C. Aubin, *Virtual reality simulator for scoliosis surgery training: Transatlantic collaborative tests*, Proceedings of IEEE International Workshop on Haptic Audio Visual Environments and Games (HAVE), Ottawa, Canada, 2008, pp. 1–6.
17. A. Sideris, M. Roussou, and A. Gaitatzes, *Virtual reconstruction of the Hellenistic Asklepieion of Messene using scientific and educational scenario-based approaches*, Imeros Journal for Culture and Technology. **4** (2004), 208–216.
18. D. Christopoulos, P. Mavridis, A. Andreadis, and J. N. Karigiannis, *Using virtual environments to tell the story: The battle of Thermopylae*, Proceedings of 3rd International Conference on Games and Virtual Worlds for Serious Applications, Athens Greece, 2011, pp. 84–91.
19. X. Chen, M. Wang, and Q. Wu, *Research and development of virtual reality game based on unreal engine*, Proceedings of 4th International Conference on Systems and Informatics (ICSAI), Hangzhou, China, 2017, pp. 1388–1393.
20. G. Rodríguez, A. Soria, and M. Campo, *Teaching scrum to software engineering students with virtual reality support*, Proceedings of the 2nd International Conference on Advances in New Technologies, Interactive Interfaces and Communicability (ADNTIIC), Berlin, Heidelberg, 2011, pp. 140–150.
21. G. Burdea and P. Coiffet, *Virtual reality technology*, 1st ed., John Wiley & Sons, New York, NY, 1994.
22. G. J. Kim, K. C. Kang, H. Kim, and J. Lee, *Software engineering of virtual worlds*, Proceedings of ACM Symposium on Virtual Reality Software and Technology (VRST), Taipei, Taiwan, 1999, pp. 131–138.
23. J. Seo and G. J. Kim, *Design for presence: A structured approach to virtual reality system design*, J. Teleoperators Virtual Environ. **11** (2002), 378–403.
24. B. Pellens, W. Bille, O. De Troyer, and F. Kleinermann, *VR-WISE: A conceptual modeling approach for virtual environments*, Proceedings of the methods and Tools for Virtual Reality Workshop (MeTo-VR), Gent, Belgium, 2005, pp. 1–10.
25. V. Tanriverdi and R. J. K. Jacob, *VRID: A design model and methodology for developing virtual reality interfaces*, Proceedings of the ACM Symposium on Virtual Reality Software and Technology (VRST), Alberta, Canada, 2001, pp. 175–182.
26. M. Scaife and Y. Rogers, *Informing the design of a virtual environment to support learning in children*, Int. J. Human-Comput. Studies. **55** (2001), no. 2, 115–143.
27. K. M. Stanney, M. Mollaghasemi, L. Reeves, R. Breaux, and D. A. Graeber, *Usability engineering of virtual environments (VEs): Identifying multiple criteria that drive effective VE system design*, Int. J. Human-Comput. Studies. **58** (2003), no. 4, 447–481.
28. A. Bezerra, M. E. Delamaro, and F. L. S. Nunes, *Automated testing of virtual reality application interfaces*, Proceedings of the Symposium on Virtual and Augmented Reality (SVR), Uberlândia, Minas Gerais, Brazil, 2011, pp. 56–65. In portuguese.
29. A. Bierbaum, P. Hartling, and C. Cruz-Neira, *Automated testing of virtual reality application interfaces*, Proceedings of the Workshop on Virtual Environments (EGVE), Goslar, Germany, 2003, pp. 107–114.
30. C. J. Chen, S. Y. Lau, K. M. Chuah, and C. S. Teh, *Group usability testing of virtual reality-based learning environments: A modified approach*, Procedia - Social Behav. Sci. **97** (2013), 691–699.
31. B. Beizer, *Software testing techniques*, 2nd ed., Intl Thomson Computer Pr(T), New York, 1990.
32. R. A. Demillo, W. M. McCracken, R. J. Martin, and J. F. Passafiume, *Software testing and evaluation*, Benjamin-Cummings Publishing Co., Inc., Workingham Menlo Park, CA, 1987.

33. L. Jacobson, *Virtual reality: A status report*, *AI Expert*. **6** (1991), no. 8, 26–33.
34. M. W. Krueger, *Artificial reality 2*, 2nd ed., Addison-Wesley, Reading, MA, 1991.
35. J. Vince, *Introduction to virtual reality*, 1st ed., Springer-Verlag, London, 2004.
36. M. Woo, J. Neider, T. Davis, and D. Shreiner, *OpenGL, programming guide*, 3rd ed., Addison-Wesley, Reading, MA, 1999.
37. B. Kuehl, K. J. Blom, and S. Beckhaus, *Generation of shadows in scene graph based VR*, *Proceedings of the International Conferences in Central Europe on Computer Graphics, Visualization and Computer Vision (WSCG)*, Plzen, Czech Republic, 2007, pp. 295–302.
38. P. G. Frankl and E. J. Weyuker, *Testing software to detect and reduce risk*, *J. Syst. Software*. **53** (2000), no. 3, 275–286.
39. P. Ammann and A. J. Offutt, *Introduction to software testing*, 2nd ed., Cambridge University Press, New York, NY, 2016.
40. P. T. Nakamoto, G. A. Carrijo, A. Cardoso, L. V. O. Lima, and E. J. Lopes, *Requirements analysis strategy for the development of augmented reality environments*, *J. Inf. Syst. Technology Manage.* **9** (2012), no. 3, 607–626. In portuguese.
41. D. A. Bowman, E. Kruijff, J. J. Laviola, and I. Poupyrev, *3D user interfaces: Theory and practice*, 1st ed., Addison-Wesley, Boston, 2004.
42. I. Sommerville, *Software engineering*, 10th ed., Pearson Education Limited, Essex, England, 2015.
43. R. Stuart, *Design of virtual environments*, McGraw-Hill, New York, 1996.
44. T. Kirner and V. Martin, *Contribution to the requirements engineering of virtual environments*, *Proceedings of the Ninth International Conference on Enterprise Information Systems (ICEIS)*, Madeira, Portugal, 2007, pp. 142–157.
45. N. Chomsky, *Three models for the description of language*, *RE Trans. Inf. Theory*. **2** (1956), no. 3, 113–124.
46. J. Allen, *Natural language understanding*, 2nd ed., Benjamin-Cummings Publishing, Menlo Park, California, 1995.
47. A. Cockburn, *Writing effective use cases*, 3rd ed., Addison-Wesley, Boston, 2001.
48. C. Heitmeyer and R. Bharadwaj, *Applying the SCR requirements method to the light control case study*, *J. Universal Comput. Sci.* **6** (2000), no. 7, 650–678.
49. D. Crystal, *Dictionary of linguistics and phonetics*, 6th ed., Wiley-Blackwell, Oxford, 2008.
50. C. J. Fillmore, *Universals in linguistic theory*, Holt, Rinehart and Wilson, New York, 1968.
51. J. Heer, S. K. Card, and J. A. Landay, *Prefuse: A toolkit for interactive information visualization*, *Proceedings of Conference on Human Factors in Computing Systems (CHI)*, New York, 2005, pp. 421–430.
52. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithm*, 3rd ed., The MIT Press, Cambridge, Massachusetts, USA, 2009.
53. T. Paar, *The definitive ANTLR reference: Building domain-specific languages*, Pragmatic Bookshelf, Dallas, Texas, 2007.
54. C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in software engineering: An introduction*, 1st ed., Springer-Verlag, Heidelberg, 2012.
55. F. D. Davis, R. P. Bagozzi, and P. R. Warshaw, *User acceptance of computer technology: A comparison of two theoretical models*, *Manage. Sci.* **35** (1989), no. 8, 982–1003.
56. R. Likert, *A technique for the measurement of attitudes*, *J. Arch. Psychology*. **22** (1932), no. 140, 1–55.
57. V. Basili and D. Weiss, *A methodology for collecting valid software engineering data.*, *IEEE Trans. Software Eng.* **10** (1986), no. 6, 728–738.
58. R. G. Hamlet, *Testing programs with the aid of a compiler*, *IEEE Trans. Software Eng.* **3** (1977), no. 4, 279–290.
59. D. Selman, *Java 3D programming*, 1st ed., The Manning, New York, USA, 2002.
60. R. F. Torres, H. H. Biscaro, L. V. Araujo, and F. L. S. Nunes, *ViMeT Game: A serious game for virtual medical training of breast biopsy*, *SBC J. 3D Interact. Syst.* **3** (2012), no. 3, 12–22.
61. T. Yeh, T. H. Chang, and R. C. Miller, *Sikuli: Using GUI screenshots for search and automation*, *Proceedings of the 22nd Annual ACM Symposium on User Interface Software and Technology (UIST)*, New York, NY, USA, 2009, pp. 183–192.
62. R. A. P. Oliveira, E. Alegroth, Z. Gao, and A. Memon, *Definition and evaluation of mutation operators for GUI-level mutation analysis*, *Proceedings of the International Workshop on Mutation Analysis*, Graz, Austria, 2015, pp. 1–10.
63. M. Florkczyk and W. Winiecki, *The parametric method for functional testing of virtual instruments*, *Proceedings of the Workshop on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications*, Sofia, Bulgaria, 2005, pp. 310–315.
64. W. Salem, *Combining VR technology and human factors methods for supporting risk analysis*, *Proceedings of the 3rd International Conference on Information and Communication Technologies: From Theory to Applications (ICTTA)*, Damascus, Syria, 2008, pp. 1–6.
65. D. Navarre, P. Palanque, J. Ladry, and E. Barboni, *ICOs: A model-based user interface description technique dedicated to interactive systems addressing usability, reliability and scalability*, *ACM Trans. Comput.-Human Interact.* **16** (2009), no. 4, 1–56.
66. E. Barboni, C. Martinie, D. Navarre, P. Palanque, and M. A. Winckler, *Bridging the gap between a behavior formal description technique and user interface description language: Enhancing ICO with a Graphical User Interface markup language*, *Sci. Comput. Program.* **86** (2014), 3–29.
67. A. Hamon, P. Palanque, M. Cronel, R. Andre, E. Barboni, and D. Navarre, *Formal modelling of dynamic instantiation of input devices and interaction techniques: Application to multi-touch interactions*, *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems (EICS)*, Rome, Italy, 2014, pp. 173–178.